# Navigation

# Navigation

**App Kit**

App Kit Overview
> **Get Started**
Installation
Supported Blockchains
Send Tokens
> **Learn More**
Adapter Setups
SDK Reference
Fees

**Bridge**
Intro to Bridging with App Kit
> **Quickstarts**
- Transfer USDC from Base to Ethereum
- Transfer USDC from Ethereum to Solana
- Transfer USDC from Ethereum to Solana with Circle Wallets
> **Customize Your Bridges**
- Collect a Bridge Fee
- Estimate Bridging Costs
- Configure Transfer Speed and Max Cost
- Specify Recipient Address
> **Learn More**
- Custom Fees
- Error Recovery and Troubleshooting

**Swap**
Intro to Swapping with App Kit
> **Quickstarts**
- Swap USDT for USDC on Ethereum
> **Customize Your Swaps**
- Estimate Swap Rate
- Collect a Swap Fee
- Set Slippage Tolerance or Stop Limit
> **Learn More**
- Custom Fees
- Error Recovery and Troubleshooting

**Kits**


App Kit Overview
- **> Get Started**
- Installation
- Supported Blockchains
- Adapter Setups
- SDK Reference
- **> Actions**
  - Send Tokens
  - **> Bridge**
  - Bridge Kit Overview
    - **> Quickstarts**
      - Transfer USDC from Base to Ethereum
      - Transfer USDC from Ethereum to Solana
      - Transfer USDC from Ethereum to Solana with Circle Wallets
    - **> Customize Your Bridges**
      - Collect a Bridge Fee
      - Estimate Bridging Costs
      - Configure Transfer Speed and Max Cost
      - Specify Recipient Address
    - **> Learn More**
      - Custom Fees
      - Error Recovery and Troubleshooting
  - **> Swap**
  - Intro to Swapping with App Kit
    - **> Quickstarts**
      - Swap USDT for USDC on Ethereum
    - **> Customize Your Swaps**
      - Estimate Swap Rate
      - Collect a Swap Fee
      - Set Slippage Tolerance or Stop Limit
    - **> Learn More**
      - Custom Fees
      - Error Recovery and Troubleshooting

Option A - App Kit + other key actions:
https://circle-167b8d39-devdocs-883-kits-0.mintlify.app/app-kit
Option B - App Kit + other kits:
https://circle-167b8d39-devdocs-883-kits-0.mintlify.app/kits

# Dev Doc Items & Priority

1. Change Overall Branding to Arc
   a. [P0] - Arc dev doc page structure change, include App Kit dev doc
   b. [P0] - Migrate bridge kit to App Kit structure under Arc
   c. [P0] - Lead in App Kit positioned as the main kit. "Bridge," "Swap," and other functionalities are included. Expand the navigation to show docs for each "verb." I will share a preview of how this looks when ready.
   d. [P0] - On the top horizontal nav, think of a different way to call it other than "Kits"
      i. [P1] - Broader:  Make the top nav more use-case based for docs holistically (chat with other tech writers)
2. App Kit Dev Doc
   a. [P0] - Add use-case quickstarts to App Kit that combine the verbs, such as transferring USDT from Ethereum to receive USDC on Arc (swap, then bridge)
      i. [P1] - Add sample use case section to emphasis on real business case (e.g. stablecoin payment, treasury management…) and provide sample code
3. Embed all the kits in docs for Circle products.
   a. [P0] - Short term: Do similarly as with CCTP and highlight kits, linking to the kits docs
      i. [P0] - CCTP page, SDK page, Get Start page?
      ii. [P1] - Broader: Embed kits more deeply throughout our docs (chat with other tech writers). Revamp the docs site holistically to position App Kit as the main entry point since it abstracts away the complexity of lower-level protocols such as CCTP and Gateway. However, if a use case requires more direct integration, those product docs are still available to view.

# App Kit Overview

# App Kit Overview

The App Kit SDK lets you build apps for common payment and liquidity workflows across blockchains. You can send, bridge, or swap tokens on supported blockchains in just a few lines of code. It provides a type-safe interface that works with Viem, Ethers, Solana Web3.js, and Circle Wallets. You can also extend the kit to support other wallet providers and frameworks.

## Key features

- **Simple setup**: Perform App Kit actions in a few lines of code.
- **Application monetization**: Collect a fee from end-users without writing new code.
- **Flexible configurations**: Specify custom RPC endpoints and wallet clients.
- **Broad compatibility**: Works with Viem, Ethers, and Solana, integrating smoothly with existing developer workflows.

## Quick look

These examples show how to perform App Kit actions in a single method call:

### Send

```
TypeScript
```

To use this code and start sending tokens from one wallet to another, see the quickstart: Send Tokens Across Wallets.

### Bridge

```
TypeScript
// Transfer 1.00 USDC from Ethereum to Solana
const result = await kit.bridge({
  from: { adapter: viemAdapter, chain: "Ethereum" },
  to: { adapter: solanaAdapter, chain: "Solana" },
```

```
  amount: "1.00",
});
```

To use this code and start bridging across blockchains, follow the quickstart for your environment:

- Transfer USDC from Arc to Base
- Transfer USDC from Ethereum to Solana
- Transfer USDC from Arc to Solana with Circle Wallets

**Swap**

```typescript
// Swap 1.00 USDT to USDC on Ethereum
const result = await kit.swap({
  from: { adapter: viemAdapter, chain: "Ethereum" },
  tokenIn: 'USDT',
  tokenOut: 'USDC',
  amount: '1.00'
});
```

To use this code and start swapping one type of token for another, see the quickstart: Swap USDT for USDC.

# Get Started (App Kit)

# Installation

# App Kit Installation

To get started with App Kit, install the core package and an adapter that fits your environment.

**Note**: App Kit requires a Kit Key obtained from the Circle Developer Console.

1. **Install App Kit**

```shell
# npm
npm install @circle-fin/stablecoin-kit

# yarn
yarn add @circle-fin/stablecoin-kit
```

2. **Install your preferred adapter**

Viem

```shell
# npm
npm install @circle-fin/adapter-viem-v2 viem

# yarn
yarn add @circle-fin/adapter-viem-v2 viem
```

Ethers

```shell
# npm
```

```
npm install @circle-fin/adapter-ethers-v6 ethers

# yarn
yarn add @circle-fin/adapter-ethers-v6 ethers
```

### Solana

```shell
# npm
npm install @circle-fin/adapter-solana @solana/web3.js

# yarn
yarn add @circle-fin/adapter-solana @solana/web3.js
```

### Circle Wallets

If you have a Circle Wallets account, you can use the Circle Wallets adapter to connect to developer-controlled wallets and Circle Contracts.

```shell
# npm
npm install @circle-fin/adapter-circle-wallets

# npm yarn
yarn add @circle-fin/adapter-circle-wallets
```

**Note**: The Circle Wallets adapter requires an API Key and Entity Secret from the Circle Developer Console. These credentials should never be exposed in the browser. For this reason, the adapter is intended for server-side use only.

# Supported Blockchains and Tokens

# App Kit Supported Blockchains

## Supported blockchains

The following table lists the mainnet and testnet blockchains that App Kit supports for the bridge, swap, and send actions.

### Actions

| Blockchain | Bridge | Swap | Send |
|---|---|---|---|
| Arc Testnet | ✅ | ❌ | ✅ |
| Arbitrum | ✅ | ✅ | ✅ |
| Arbitrum Sepolia | ✅ | ❌ | ✅ |
| Avalanche | ✅ | ✅ | ✅ |
| Avalanche Fuji | ✅ | ❌ | ✅ |
| Base | ✅ | ✅ | ✅ |
| Base Sepolia | ✅ | ❌ | ✅ |
| Codex | ✅ | ❌ | ✅ |
| Codex Testnet | ✅ | ❌ | ✅ |
| Ethereum | ✅ | ✅ | ✅ |
| Ethereum Sepolia | ✅ | ❌ | ✅ |
| HyperEVM | ✅ | ✅ | ✅ |
| HyperEVM Testnet | ✅ | ❌ | ✅ |
| Ink | ✅ | ✅ | ✅ |
| Ink Testnet | ✅ | ❌ | ✅ |
| Linea | ✅ | ✅ | ✅ |

| Blockchain | Bridge | Swap | Send |
|---|---|---|---|
| Linea Sepolia | ✅ | ❌ | ✅ |
| Monad | ✅ | ✅ | ✅ |
| Monad Testnet | ✅ | ❌ | ✅ |
| OP Mainnet | ✅ | ✅ | ✅ |
| OP Sepolia | ✅ | ❌ | ✅ |
| Plume | ✅ | ✅ | ✅ |
| Plume Testnet | ✅ | ❌ | ✅ |
| Polygon PoS | ✅ | ✅ | ✅ |
| Polygon PoS Amoy | ✅ | ❌ | ✅ |
| Sei | ✅ | ✅ | ✅ |
| Sei Testnet | ✅ | ❌ | ✅ |
| Solana | ✅ | ✅ | ✅ |
| Solana Devnet | ✅ | ❌ | ✅ |
| Sonic | ✅ | ✅ | ✅ |
| Sonic Testnet | ✅ | ❌ | ✅ |
| Unichain | ✅ | ✅ | ✅ |
| Unichain Sepolia | ✅ | ❌ | ✅ |
| World Chain | ✅ | ✅ | ✅ |
| World Chain Sepolia | ✅ | ❌ | ✅ |
| XDC | ✅ | ✅ | ✅ |
| XDC Apothem | ✅ | ❌ | ✅ |

## Adapters

The following table lists the mainnet and testnet blockchains that App Kit supports for chain adapters (Viem, Ethers, Solana) and the Circle Wallets adapter.

| Blockchain | Chain adapters | Circle Wallets adapter |
|---|---|---|
| Arbitrum | ✅ | ✅ |
| Arbitrum Sepolia | ✅ | ✅ |
| Avalanche | ✅ | ✅ |
| Avalanche Fuji | ✅ | ✅ |
| Base | ✅ | ✅ |
| Base Sepolia | ✅ | ✅ |
| Codex | ✅ | ❌ |
| Codex Testnet | ✅ | ❌ |
| Ethereum | ✅ | ✅ |
| Ethereum Sepolia | ✅ | ✅ |
| HyperEVM | ✅ | ❌ |
| HyperEVM Testnet | ✅ | ❌ |
| Ink | ✅ | ❌ |
| Ink Testnet | ✅ | ❌ |
| Linea | ✅ | ❌ |
| Linea Sepolia | ✅ | ❌ |
| Monad | ✅ | ❌ |
| Monad Testnet | ✅ | ❌ |
| OP Mainnet | ✅ | ✅ |

| Blockchain | Chain adapters | Circle Wallets adapter |
|---|---|---|
| OP Sepolia | ✅ | ✅ |
| Plume | ✅ | ❌ |
| Plume Testnet | ✅ | ❌ |
| Polygon PoS | ✅ | ✅ |
| Polygon PoS Amoy | ✅ | ✅ |
| Sei | ✅ | ❌ |
| Sei Testnet | ✅ | ❌ |
| Solana | ✅ | ✅ |
| Solana Devnet | ✅ | ✅ |
| Sonic | ✅ | ❌ |
| Sonic Testnet | ✅ | ❌ |
| Unichain | ✅ | ✅ |
| Unichain Sepolia | ✅ | ✅ |
| World Chain | ✅ | ❌ |
| World Chain Sepolia | ✅ | ❌ |
| XDC | ✅ | ❌ |
| XDC Apothem | ✅ | ❌ |

## Supported tokens

App Kit works with any token on supported blockchains. It provides aliases for the most common tokens so you don't need to track their contract addresses. For any other token, you can supply the contract address. The following aliases are available:

- `USDC`
- `EURC`
- `USDT`
- `USDe`
- `DAI`
- `PYUSD`
- `NATIVE`: Uses the blockchain's native token.

# Send Tokens Across Wallets

# Quickstart: Send Tokens Across Wallets

This quickstart helps you write a script to send tokens from your wallet to another on Ethereum.

## Prerequisites

-

## Step 1. Set up the project

This step shows you how to prepare your project and environment.

### 1.1. Set up your development environment

Create a new directory and install App Kit and its dependencies:

```shell
# Set up your directory and initialize a Node.js project
mkdir app-kit-send
cd app-kit-send
npm init -y

# Install App Kit and tools
npm install @circle-fin/stablecoin-kit @circle-fin/adapter-viem-v2 viem
typescript tsx dotenv
```

### 1.2. Configure environment variables

Create a .env file in the project directory and add the required keys:

- Replace `{YOUR_PRIVATE_KEY}` with the private key for your Ethereum wallet.
- Replace `{YOUR_KIT_KEY}` with the Kit key that you generated from the Circle Developer Console.

```shell
cat <<'EOF' > .env
```

```
PRIVATE_KEY={YOUR_PRIVATE_KEY}
KIT_KEY={YOUR_KIT_KEY}
EOF
```

## Step 2. Perform the send

This step shows you how to set up your script, send tokens from your wallet to another wallet, and check the result.

### 2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script. It sends 100 USDC from one wallet to another:

```
TypeScript
```

### 2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```
TypeScript
npx tsx index.ts
```

### 2.3. Verify the transfer

After the script finishes, find the returned steps array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify the amount matches how much USDC was received.

The following code is an example of how the result of a successful send might look in the terminal output. The values are used in this example only and are not a real transaction:

```
TypeScript
```

# Quickstart: Convert USDT on Solana to USDC on Arc

# Quickstart: Convert USDT on Solana to USDC on Arc

This quickstart helps you write a script to convert USDT on Ethereum to USDC on Arc. This use case involves the following steps:

1. Swap USDT to USDC on Ethereum
2. Bridge USDC from Ethereum to Arc
3. (Optional) Send USDC to another address on Arc

## Prerequisites

Before you begin, ensure that you've:

- Installed Node.js v22+ and npm.
- Created an Ethereum mainnet wallet and funded it with USDT and ETH tokens.
- Obtained a Kit key from the Circle Developer Console.

## Step 1. Set up the project

This step shows you how to prepare your project and environment.

### 1.1. Set up your development environment

Create a new directory and install Swap Kit and its dependencies:

```shell
# Set up your directory and initialize a Node.js project
mkdir stablecoin-kit-quickstart-USDT-to-USDC
cd stablecoin-kit-quickstart-USDC-to-USDT
npm init -y

# Install Stablecoin Kit and tools
npm install @circle-fin/stablecoin-kit @circle-fin/adapter-viem-v2 viem
typescript tsx dotenv
```

### 1.2. Configure environment variables

Create a .env file in the project directory and add the required keys:

- Replace `{YOUR_PRIVATE_KEY}` with the private key for your Ethereum wallet.
- Replace `{YOUR_KIT_KEY}` with the Kit key that you generated from the Circle Developer Console.

```shell
cat <<'EOF' > .env
PRIVATE_KEY={YOUR_PRIVATE_KEY}
KIT_KEY={YOUR_KIT_KEY}
EOF
```

## Step 2. Setup the script to swap, bridge and (optional) send

This step shows you how to set up your script to execute the flows, and check the result.

### 2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script.

1. It firstly swaps 1 USDT to USDC on Ethereum
2. Bridge 1 USDC from Ethereum to Arc
3. (Optinoal) Send 1 USDC to another address on Arc

```typescript
// Import Stablecoin Kit and its dependencies
import "dotenv/config";
import { StablecoinKit } from "@circle-fin/stablecoin-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { inspect } from "util";

// Initialize the SDK
const kit = new StablecoinKit();

const swapUSDTtoUSDC = async (): Promise<void> => {
  try {
    // Initialize the adapter
    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
```

```
    });

    console.log("---------------Starting Swapping---------------");

    const result = await kit.swap({
      from: { adapter, chain: "Ethereum" },
      tokenIn: "USDT",
      tokenOut: "USDC",
      amount: "1.00",
      config: {
         kitKey: process.env.KIT_KEY as string
      }
    });

    console.log("RESULT", inspect(result, false, null, true));


    console.log("---------------Starting Bridging---------------");

    const result = await kit.bridge({
      from: { adapter, chain: "Ethereum" },
      to: { adapter, chain: "Arc" },
      amount: "1.00",
    });

    console.log("RESULT", inspect(result, false, null, true));



    console.log("---------------Starting Sending---------------");
    //Send to a different wallet address

  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void swapUSDTtoUSDC();
```

## 2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```typescript
npx tsx index.ts
```

## 2.3. Verify the result

After the script finishes, find the returned steps array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify the amount matches how much USDC was received.

The following code is an example of how the result of a successful swap might look in the terminal output. The values are used in this example only and are not a real transaction:

```typescript
{
  tokenIn: 'USDT',
  tokenOut: 'USDC',
  chain: {
    type: 'evm',
    chain: 'Ethereum',
    name: 'Ethereum',
    title: 'Ethereum Mainnet',
    nativeCurrency: { name: 'Ether', symbol: 'ETH', decimals: 18 },
    chainId: 1,
    isTestnet: false,
    explorerUrl: 'https://etherscan.io/tx/{hash}',
    rpcEndpoints: [ 'https://eth.merkle.io', 'https://ethereum.publicnode.com'
],
    eurcAddress: '0x1aBaEA1f7C830bD89Acc67eC4af516284b1bC33c',
    usdcAddress: '0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48',
    usdtAddress: '0xdac17f958d2ee523a2206206994597c13d831ec7',
    cctp: {
      domain: 0,
      contracts: {
        v1: {
          type: 'split',
          tokenMessenger: '0xbd3fa81b58ba92a82136038b25adec7066af3155',
          messageTransmitter: '0x0a992d191deec32afe36203ad87d7d289a738f81',
          confirmations: 65
```

```
        },
        v2: {
          type: 'split',
          tokenMessenger: '0x28b5a0e9C621a5BadaA536219b3a228C8168cf5d',
          messageTransmitter: '0x81D40F21F12A8F0E3252Bccb954D722d4c464B64',
          confirmations: 65,
          fastConfirmations: 2
        }
      }
    },
    kitContracts: {
      bridge: '0xB3FA262d0fB521cc93bE83d87b322b8A23DAf3F0',
      adapter: '0x7FB8c7260b63934d8da38aF902f87ae6e284a845'
    }
  },
  amount: '1.00',
  fromAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  toAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  txHash: '0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99d5cccf205897',
  explorerUrl:
'https://etherscan.io/tx/0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99
d5cccf205897',
  fees: [ { token: 'USDT', amount: '0.001', type: 'provider' } ],
  config: {
    kitKey:
'KIT_KEY:fdd99fdbdb3c87b9b3f7b29d6cc17b6e:cc8777b4e419eb6dda9f0de932cf0bb8',
  }
}
```

# Learn More (App Kit)

# Adapter Setups

> ## Documentation Index
> Fetch the complete documentation index at: https://developers.circle.com/llms.txt
> Use this file to discover all available pages before exploring further.

# Adapter Setups

App Kit supports these adapter types:

* **Chain client adapters** abstract away the complexity of different blockchain architectures. Each adapter manages blockchain-specific fund orchestration operations.
* The **Circle Wallet adapter** is available if you already use Circle's [developer-controlled wallets](/wallets/dev-controlled). It uses your Circle Wallets account to let you transfer between [supported blockchains](/app-kit/references/supported-blockchains).

## Chain client adapter setups

App Kit works with these client adapter libraries:

* [`viem`](https://viem.sh/) v2 for EVM-compatible blockchains
* [`ethers`](https://ethers.org/) v6 for EVM-compatible blockchains
* [`solana`](https://solana.com/) for the Solana blockchain

You only need to set up with one EVM adapter, either Viem or Ethers. Add the Solana adapter to your setup only if you plan to transfer to or from Solana.

The following table shows ways to set up chain client adapters and when to use each setup.

| Setup method | Description | Use case |
| --- | --- | --- |
| [Standard](#standard-setup) | Standard setup that creates an adapter from a private wallet key. Uses default built-in public RPC endpoints and factory functions. | Testing environments that need a quick start without custom RPC configuration. |

| [Custom RPC](#custom-rpc) | Private key setup that lets you configure an RPC endpoint. | Production deployments that require reliable RPC providers. |
| [Browser wallet](#browser-wallet) | Uses an injected wallet provider, such as MetaMask or Phantom, to create an adapter in the browser. | Browser-based apps that rely on user wallet providers. |

### Standard setup

This setup is the fastest way to start. Create one adapter from your wallet private key to transfer tokens between EVM-compatible blockchains. Add the Solana adapter to transfer tokens to or from Solana.

This setup uses public RPC endpoints and factory functions. For production, you should configure a [custom RPC](#custom-rpc). Public connections have rate limits and might be slow.

Select the tab to see code examples for your adapter choice.

<Tabs>
  <Tab title="Viem">
    This code block initializes the Viem adapter to transfer tokens between EVM-compatible chains:

    ```typescript TypeScript theme={null}
    import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";

    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });
    ```
  </Tab>

  <Tab title="Ethers">
    This code block initializes the Ethers adapter to transfer tokens between EVM-compatible chains:

    ```typescript TypeScript theme={null}
    import { createEthersAdapterFromPrivateKey } from "@circle-fin/adapter-ethers-v6";
    ```

```
    const adapter = createEthersAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });
```
  </Tab>

  <Tab title="Solana">
    This code block initializes the Solana adapter to bridge on Solana:

```typescript TypeScript theme={null}
    import { createSolanaKitAdapterFromPrivateKey } from
"@circle-fin/adapter-solana-kit";

    // Solana accepts Base58, Base64, or JSON array format private keys
    const adapter = createSolanaKitAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });
```
  </Tab>
</Tabs>

### Custom RPC

You can replace the public RPC from the [standard setup](#standard-setup) with your own connection. For production, you should use a paid service like [Alchemy](https://www.alchemy.com/) or [QuickNode](https://www.quicknode.com/). These services are more reliable than free connections.

The following code shows how to set up your own RPC. Pick your adapter to see the code.

<Tabs>
  <Tab title="Viem">
    To use your own connection, replace the default one and add your custom RPC endpoints. Map each chain to its RPC endpoint so one adapter can work across multiple chains. This example uses Alchemy connections:

```typescript TypeScript theme={null}
    import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
    import { EthereumSepolia, ArcTestnet } from "@circle-fin/bridge-kit/chains";
```

```
import { createPublicClient, http } from "viem";

// Map RPC endpoints by chain name
const RPC_BY_CHAIN_NAME: Record<string, string> = {
  [EthereumSepolia.name]:
`https://eth-sepolia.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`,
  [ArcTestnet.name]:
`https://arc-testnet.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`,
};

// Create an adapter
const adapter = createViemAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as string,
  // Replace the default connection
  getPublicClient: ({ chain }) => {
    const rpcUrl = RPC_BY_CHAIN_NAME[chain.name];
    if (!rpcUrl) {
      throw new Error(`No RPC configured for chain: ${chain.name}`);
    }
    return createPublicClient({
      chain,
      transport: http(rpcUrl, {
        retryCount: 3,
        timeout: 10000,
      }),
    });
  },
});
```
</Tab>

<Tab title="Ethers">
To use your own connection, replace the default one and add your custom RPC endpoints. Map each chain to its RPC endpoint so one adapter can work across multiple chains. This example uses Alchemy connections:

```typescript TypeScript theme={null}
import { createEthersAdapterFromPrivateKey } from "@circle-fin/adapter-ethers-v6";
import { EthereumSepolia, ArcTestnet } from "@circle-fin/bridge-kit/chains";
import { JsonRpcProvider } from "ethers";
```

```
// Map RPC endpoints by chain name
const RPC_BY_CHAIN_NAME: Record<string, string> = {
  [EthereumSepolia.name]:
`https://eth-sepolia.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`,
  [ArcTestnet.name]:
`https://arc-testnet.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`,
};

// Create an adapter
const adapter = createEthersAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as string,
  // Replace the default connection
  getProvider: ({ chain }) => {
    const rpcUrl = RPC_BY_CHAIN_NAME[chain.name];
    if (!rpcUrl) {
      throw new Error(`No RPC configured for chain: ${chain.name}`);
    }
    return new JsonRpcProvider(rpcUrl);
  },
});
```
</Tab>

<Tab title="Solana">
To use your own connection, create a custom connection to your RPC endpoint. Point to that connection when making a Solana adapter. This example creates a custom connection using Alchemy:

```typescript TypeScript theme={null}
import { createSolanaKitAdapterFromPrivateKey } from
"@circle-fin/adapter-solana-kit";
import { createSolanaRpc } from "@solana/kit";

// Create an adapter
const adapter = createSolanaKitAdapterFromPrivateKey({
  privateKey: process.env.SOLANA_PRIVATE_KEY as string,
  // Replace the default connection
  getRpc: () =>
    // Use an Alchemy connection
```

```
      createSolanaRpc(
        `https://solana-devnet.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`,
      ),
  });
```
</Tab>
</Tabs>

### Browser wallet

You can create an adapter from browser wallet apps like [MetaMask](https://metamask.io/) or [Phantom](https://phantom.com/).

The following code creates an adapter from a browser wallet. Pick your adapter to see the code.

<Tabs>
<Tab title="Viem">
```typescript TypeScript theme={null}
import { createViemAdapterFromProvider } from "@circle-fin/adapter-viem-v2";
import type { EIP1193Provider } from "viem";

declare global {
  interface Window {
    ethereum?: EIP1193Provider;
  }
}

// Check if wallet provider is available
if (!window.ethereum) {
  throw new Error("No wallet provider found");
}

const adapter = await createViemAdapterFromProvider({
  provider: window.ethereum,
});
```
</Tab>

<Tab title="Ethers">

```typescript TypeScript theme={null}
import { createEthersAdapterFromProvider } from "@circle-fin/adapter-ethers-v6";
import type { Eip1193Provider } from "ethers";

declare global {
  interface Window {
    ethereum?: Eip1193Provider;
  }
}

// Check if wallet is installed
if (!window.ethereum) {
  throw new Error("No wallet provider found");
}

const adapter = await createEthersAdapterFromProvider({
  provider: window.ethereum,
});
```
</Tab>

<Tab title="Solana">
```typescript TypeScript theme={null}
import {
  createSolanaKitAdapterFromProvider,
  SolanaKitWalletProvider,
} from "@circle-fin/adapter-solana-kit";

declare global {
  interface Window {
    solana?: SolanaKitWalletProvider;
  }
}

if (window.solana) {
  const adapter = await createSolanaKitAdapterFromProvider({
    provider: window.solana,
  });
}
```

</Tab>
</Tabs>

## Circle Wallets adapter setup

<Note>
**Note**: This adapter is for server-side applications only. It requires private keys and entity secrets that must never be exposed to browsers.
</Note>

You can use the Circle Wallets adapter if you already manage wallets through Circle. Suited for enterprise and backend applications, it uses Circle's [developer-controlled wallets](/wallets/dev-controlled) and [Circle Contracts](/contracts) so you can transfer between [supported blockchains](/bridge-kit/references/supported-blockchains) without managing private keys yourself.

The Circle Wallets adapter requires a [Circle API Key](/w3s/circle-developer-account#creating-an-api-key-for-developer-services) and [Entity Secret](/wallets/dev-controlled/register-entity-secret) obtained from the Circle Developer Console.

**API Key format:**

* Environment-prefixed: `TEST_API_KEY:abc123:def456` or `LIVE_API_KEY:xyz:uvw`
* Base64 encoded: standard Base64 string

**Entity Secret format:**

* 64 lowercase alphanumeric characters

This code block initializes the Circle Wallets adapter:

```typescript Typescript theme={null}
import { createCircleWalletsAdapter } from "@circle-fin/adapter-circle-wallets";

// Initialize adapter with Circle credentials (server-side only)
const adapter = createCircleWalletsAdapter({
```

```
  apiKey: process.env.CIRCLE_API_KEY!, // Format: TEST_API_KEY:abc:def or Base64
  entitySecret: process.env.CIRCLE_ENTITY_SECRET!, // Format: 64 lowercase alphanumeric chars
});
```

# SDK Reference

# Stablecoin Kit SDK Reference

# Stablecoin Kit SDK Reference

This reference guide describes the public interfaces, methods, and types available in the Stablecoin Kit SDK. The full SDK is [available on GitHub](https://github.com/circlefin/stablecoin-kit).

## StablecoinKit Class

The `StablecoinKit` is how you'll perform all stablecoin operations including cross-chain bridging, same-chain swaps, token transfers, and fee estimation. It also enables you to add event listeners for bridge operations.

### constructor(config?)

Creates a new `StablecoinKit` instance.

```typescript
constructor(config?: StablecoinKitConfig)
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| config | [StablecoinKitConfig](#stablecoinkitconfig) | Optional configuration for developer fees and context customization |

**Usage**

```typescript
import { StablecoinKit } from '@circle-fin/stablecoin-kit'

// Standard setup
const kit = new StablecoinKit()

// Setup with developer fee hooks
const kit = new StablecoinKit({
```

```
  developerFee: {
    getFee: (params) => 1000000n, // 1 USDC fee
    getFeeRecipient: (chain) => '0x1234567890123456789012345678901234567890'
  }
})
```

### StablecoinKitConfig

Configuration options for the StablecoinKit.

```typescript
type StablecoinKitConfig = CreateContextParams & {
  developerFee?: Partial<DeveloperFeeHooks>
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| developerFee | [DeveloperFeeHooks](#developerfeehooks) | Optional developer fee hooks for custom fee collection (optional) |

### DeveloperFeeHooks

Interface for custom developer fee calculation and recipient configuration.

```typescript
interface DeveloperFeeHooks {
  getFee: (params: BridgeParams) => Promise<bigint> | bigint
  getFeeRecipient: (chain: ChainDefinition) => Promise<string> | string
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|

| getFee | function | Returns the developer fee in USDC's smallest units (e.g., 6 decimals). For example, return `1000000n` for 1 USDC. |
| getFeeRecipient | function | Returns the fee recipient address for the given chain. Must be a valid address for the source chain. |

**Usage Example**

```typescript
const kit = new StablecoinKit({
  developerFee: {
    getFee: (params) => {
      // Charge 0.5 USDC flat fee
      return 500000n
    },
    getFeeRecipient: (chain) => {
      // Return appropriate address based on chain type
      if (chain.type === 'solana') {
        return 'EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v'
      }
      return '0x1234567890123456789012345678901234567890'
    }
  }
})
```

---

## Methods

### bridge(params: BridgeParams)

Executes a cross-chain USDC bridge transfer using Circle's Cross-Chain Transfer Protocol (CCTP).

```typescript
async bridge(params: BridgeParams): Promise<BridgeResult>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [BridgeParams](#bridgeparams) | Bridge transfer parameters |

**Returns**

[BridgeResult](#bridgeresult)

**Usage Example**

```typescript
import { StablecoinKit } from '@circle-fin/stablecoin-kit'
import { createAdapterFromPrivateKey } from '@circle-fin/adapter-viem-v2'

const kit = new StablecoinKit()

const adapter = createAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as `0x${string}`,
})

const result = await kit.bridge({
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.50',
  token: 'USDC'
})

console.log('Bridge completed:', result.state)
console.log('Transaction steps:', result.steps)
```

---

### estimateBridge(params: BridgeParams)

Estimates costs before executing a bridge transfer.

```typescript
async estimateBridge(params: BridgeParams): Promise<EstimateResult>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [BridgeParams](#bridgeparams) | Bridge transfer parameters |

**Returns**

[EstimateResult](#estimateresult)

**Usage Example**

```typescript
// Estimate costs before bridging
const estimate = await kit.estimateBridge({
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.50',
  token: 'USDC'
})

console.log('Gas fees:', estimate.gasFees)
console.log('Protocol fees:', estimate.fees)

// Check if the fee is acceptable before proceeding
const protocolFee = estimate.fees.find((f) => f.type === 'provider')?.amount
if (protocolFee == null || parseFloat(protocolFee) < 0.5) {
  const result = await kit.bridge({
    from: { adapter, chain: 'Ethereum' },
    to: { adapter, chain: 'Base' },
    amount: '100.50'
  })
  console.log('Bridge completed:', result.state)
} else {
  console.log('Fee too high:', protocolFee)
}
```

---

### send(params: SendParams)

Executes a same-chain token transfer for USDC, USDT, or native tokens.

```typescript
async send(params: SendParams): Promise<BridgeStep>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [SendParams](#sendparams) | Send transfer parameters |

**Returns**

[BridgeStep](#bridgestep)

**Usage Example**

```typescript
// Send USDC to another adapter
const result = await kit.send({
  from: { adapter, chain: 'Polygon' },
  to: destinationAdapter,
  amount: '50.00',
  token: 'USDC'
})

console.log('Send completed:', result.txHash)
console.log('Explorer URL:', result.explorerUrl)

// Send to an explicit address
const result = await kit.send({
  from: { adapter, chain: 'Ethereum' },
  to: '0x742d35Cc6634C0532925a3b844Bc454e4438f44e',
  amount: '25.00',
  token: 'USDT'
})
```

```

---

### estimateSend(params: SendParams)

Estimates gas fees for a send operation without executing.

```typescript
async estimateSend(params: SendParams): Promise<EstimatedGas>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [SendParams](#sendparams) | Send transfer parameters |

**Returns**

[EstimatedGas](#estimatedgas)

**Usage Example**

```typescript
const estimate = await kit.estimateSend({
  from: { adapter, chain: 'Polygon' },
  to: destinationAdapter,
  amount: '100.00',
  token: 'USDC'
})

console.log('Estimated gas:', estimate.gas.toString())
console.log('Gas price:', estimate.gasPrice.toString())
console.log('Total fee:', estimate.fee)
```

---

### swap(params: SwapParams)
```

Executes a same-chain token swap between USDC, USDT, and native tokens.

```typescript
async swap(params: SwapParams): Promise<SwapResult>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [SwapParams](#swapparams) | Swap operation parameters |

**Returns**

[SwapResult](#swapresult)

**Usage Example**

```typescript
const result = await kit.swap({
  from: { adapter, chain: 'Ethereum' },
  tokenIn: 'USDC',
  tokenOut: 'USDT',
  amount: '100.50',
  config: {
    slippageBps: 300, // 3% slippage tolerance
    allowanceStrategy: 'permit',
    kitKey: 'KIT_KEY:id:secret'
  }
})

console.log('Swap completed:', result.txHash)
console.log('Explorer URL:', result.explorerUrl)
console.log('Fees:', result.fees)
```

---

### estimateSwap(params: SwapParams)

Estimates output and fees for a swap operation without executing.

```typescript
async estimateSwap(params: SwapParams): Promise<SwapEstimate>
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| params | [SwapParams](#swapparams) | Swap operation parameters |

**Returns**

[SwapEstimate](#swapestimate)

**Usage Example**

```typescript
const estimate = await kit.estimateSwap({
  from: { adapter, chain: 'Ethereum' },
  tokenIn: 'USDC',
  tokenOut: 'USDT',
  amount: '100.50',
  config: {
    slippageBps: 300,
    kitKey: 'KIT_KEY:id:secret'
  }
})

console.log('Minimum output (stop limit):', estimate.stopLimit)
console.log('Estimated output:', estimate.estimatedOutput)
console.log('Fees:', estimate.fees)
```

---

### getSupportedChains(operationType?)

Returns a list of blockchains supported by StablecoinKit operations.

```typescript
getSupportedChains(operationType?: OperationType): ChainDefinition[]
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| operationType | `'bridge'` \| `'swap'` | Optional filter for operation type. If omitted, returns all supported chains. |

**Returns**

`ChainDefinition[]` - An array of chain definitions, each containing:
- `chain` - Blockchain identifier enum value
- `name` - Human-readable name for UI displays
- `type` - Blockchain type (`'evm'` or `'solana'`)
- `chainId` - Network ID (EVM chains only)
- `usdcAddress` - USDC token contract address
- `usdtAddress` - USDT token contract address (if supported)
- `cctp` - CCTP V2 configuration including domain ID and contract addresses

**Return Example**

```typescript
const chains = kit.getSupportedChains()
console.log(chains)
[
  {
    chain: "Ethereum",
    name: "Ethereum",
    type: "evm",
    chainId: 1,
    usdcAddress: "0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48",
    usdtAddress: "0xdac17f958d2ee523a2206206994597c13d831ec7",
    cctp: {
      domain: 0,
      contracts: {
```

```
      v2: {
        tokenMessenger: "0x28b5a0e9C621a5BadaA536219b3a228C8168cf5d",
        messageTransmitter: "0x81D40F21F12A8F0E3252Bccb954D722d4c464B64"
      }
    }
  }
},
{
  chain: "Solana",
  name: "Solana",
  type: "solana",
  usdcAddress: "EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v",
  cctp: {
    domain: 5,
    contracts: {
      tokenMessengerMinter:
"CCTPmbSD7gX1bxKPAmg77w8oFzNFpaQiQUWD43TKaecd",
      messageTransmitter:
"CCTPiPYPc6AsJuwueEnWgSgucamXDZwBd53dQ11YiKX3"
    }
  }
}
]
```

**Usage Example**

```typescript
import { StablecoinKit } from '@circle-fin/stablecoin-kit'

const kit = new StablecoinKit()

// Get all supported chains
const allChains = kit.getSupportedChains()
console.log(`Total supported chains: ${allChains.length}`)

// Get bridge-specific chains
const bridgeChains = kit.getSupportedChains('bridge')
console.log('Bridge chains:', bridgeChains.map(c => c.name))
```

```
// Get swap-specific chains
const swapChains = kit.getSupportedChains('swap')
console.log('Swap chains:', swapChains.map(c => c.name))
```

---

### on(action, handler)

Registers an event handler for a specific StablecoinKit action.

```typescript
on<K extends StablecoinKitActionName>(
  action: K,
  handler: (payload: StablecoinKitActions[K]) => void
): void

on(
  action: '*',
  handler: (payload: StablecoinKitActions[keyof StablecoinKitActions]) => void
): void
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| action | string | [Event type](#event-types) (prefixed with `bridge.`) or `*` for all events |
| handler | function | Callback function invoked when the action occurs |

**Returns**

`void`

**Usage Example**

```typescript
// Listen to specific bridge events
kit.on('bridge.approve', (payload) => {
  console.log('Approval completed:', payload.values.txHash)
```

```
})

kit.on('bridge.burn', (payload) => {
  console.log('Burn completed:', payload.values.txHash)
})

kit.on('bridge.mint', (payload) => {
  console.log('Mint completed:', payload.values.txHash)
})

// Listen to all events with wildcard
kit.on('*', (event) => {
  console.log(`[${event.method}] Progress:`, event.values)
})
```

---

### off(action, handler)

Removes a previously registered event handler.

```typescript
off<K extends StablecoinKitActionName>(
  action: K,
  handler: (payload: StablecoinKitActions[K]) => void
): void

off(
  action: '*',
  handler: (payload: StablecoinKitActions[keyof StablecoinKitActions]) => void
): void
```

**Parameters**

| Name | Type | Description |
|------|------|-------------|
| action | string | [Event type](#event-types) (prefixed with `bridge.`) or `*` for all events |
| handler | function | The exact handler function reference to remove |

**Returns**

`void`

**Usage Example**

```typescript
const burnHandler = (event) => console.log('Burn:', event.values.txHash)

// Register handler
kit.on('bridge.burn', burnHandler)

// Execute bridge operation
await kit.bridge({ ... })

// Later, remove handler
kit.off('bridge.burn', burnHandler)
```

---

## Method Parameters

### BridgeParams

Parameters for executing a cross-chain USDC bridge transfer.

```typescript
interface BridgeParams<
  TFromAdapterCapabilities extends AdapterCapabilities = AdapterCapabilities,
  TToAdapterCapabilities extends AdapterCapabilities = AdapterCapabilities,
> {
  from: AdapterContext<TFromAdapterCapabilities>
  to: BridgeDestination<TToAdapterCapabilities>
  amount: string
  config?: BridgeConfig
  token?: 'USDC'
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| from | [AdapterContext](#adaptercontext) | Source adapter and chain context |
| to | [BridgeDestination](#bridgedestination) | Destination adapter, chain, and optional recipient address |
| amount | string | Amount to transfer in human-readable format (e.g., `'100.50'`) |
| config | [BridgeConfig](#bridgeconfig) | Optional bridge configuration (transfer speed, fees) |
| token | `'USDC'` | Token to transfer (optional, defaults to `'USDC'`) |

**Usage Example**

```typescript
import { createAdapterFromPrivateKey } from '@circle-fin/adapter-viem-v2'

const adapter = createAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as `0x${string}`,
})

// Basic transfer with defaults
const basicParams: BridgeParams = {
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.50'
}

// Transfer with speed configuration
const fastParams: BridgeParams = {
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Polygon' },
  amount: '50.00',
  config: { transferSpeed: 'FAST' },
  token: 'USDC'
}

// Transfer to a custom recipient address
const customRecipient: BridgeParams = {
```

```
  from: { adapter, chain: 'Base' },
  to: {
    adapter,
    chain: 'Arbitrum',
    recipientAddress: '0x742d35Cc6634C0532925a3b844Bc454e4438f44e'
  },
  amount: '25.00'
}
```

---

### SendParams

Parameters for executing a same-chain token transfer.

```typescript
interface SendParams {
  from: AdapterContext
  to: Adapter | string
  amount: string
  token?: 'USDC' | 'USDT' | 'NATIVE'
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| from | [AdapterContext](#adaptercontext) | Source adapter context (wallet and chain) |
| to | Adapter \| string | Destination adapter or explicit recipient address |
| amount | string | Amount to transfer in human-readable format |
| token | `'USDC'` \| `'USDT'` \| `'NATIVE'` | Token to transfer (optional, defaults to `'USDC'`) |

**Usage Example**

```typescript
// Send to another adapter
const sendToAdapter: SendParams = {
```

```
  from: { adapter: sourceAdapter, chain: 'Ethereum' },
  to: destinationAdapter,
  amount: '50.00',
  token: 'USDC'
}

// Send to an explicit address
const sendToAddress: SendParams = {
  from: { adapter, chain: 'Polygon' },
  to: '0x742d35Cc6634C0532925a3b844Bc454e4438f44e',
  amount: '100.00',
  token: 'USDT'
}

// Send native token
const sendNative: SendParams = {
  from: { adapter, chain: 'Ethereum' },
  to: '0x742d35Cc6634C0532925a3b844Bc454e4438f44e',
  amount: '0.5',
  token: 'NATIVE'
}
```

---

### SwapParams

Parameters for executing a same-chain token swap.

```typescript
interface SwapParams<
  TFromAdapterCapabilities extends AdapterCapabilities = AdapterCapabilities,
> {
  from: AdapterContext<TFromAdapterCapabilities>
  tokenIn: SupportedToken
  tokenOut: SupportedToken
  amount: string
  config?: SwapConfig
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| from | [AdapterContext](#adaptercontext) | Source adapter context (wallet and chain) |
| tokenIn | [SupportedToken](#supportedtoken) | Input token (`'USDC'`, `'USDT'`, or `'NATIVE'`) |
| tokenOut | [SupportedToken](#supportedtoken) | Output token (`'USDC'`, `'USDT'`, or `'NATIVE'`) |
| amount | string | Amount of tokenIn to swap in human-readable format |
| config | [SwapConfig](#swapconfig) | Optional swap configuration |

**Usage Example**

```typescript
// Basic swap from USDC to USDT
const swapParams: SwapParams = {
  from: { adapter, chain: 'Ethereum' },
  tokenIn: 'USDC',
  tokenOut: 'USDT',
  amount: '100.50'
}

// Swap with slippage and permit
const swapWithConfig: SwapParams = {
  from: { adapter, chain: 'Polygon' },
  tokenIn: 'USDT',
  tokenOut: 'USDC',
  amount: '50.00',
  config: {
    slippageBps: 300, // 3% slippage
    allowanceStrategy: 'permit',
    kitKey: 'KIT_KEY:id:secret'
  }
}

// Swap stablecoin to native token
const swapToNative: SwapParams = {
  from: { adapter, chain: 'Ethereum' },
```

```
  tokenIn: 'USDC',
  tokenOut: 'NATIVE',
  amount: '100.00'
}
```

---

### SwapConfig

Configuration options for swap operations.

```typescript
interface SwapConfig {
  allowanceStrategy?: 'permit' | 'approve'
  slippageBps?: number
  stopLimit?: string
  customFee?: {
    value?: string
    recipientAddress?: string
  }
  kitKey?: string
}
```

**Properties**

| Name | Type | Description |
|------|------|----------|
| allowanceStrategy | `'permit'` \| `'approve'` | Token allowance strategy (optional, defaults to `'permit'` with fallback to `'approve'`) |
| slippageBps | number | Maximum slippage in basis points. 1 BPS = 0.01%, so 300 = 3% (optional, defaults to 300) |
| stopLimit | string | Minimum acceptable output amount in human-readable format (optional) |
| customFee | object | Custom fee configuration with value and recipient address (optional) |
| kitKey | string | Kit identifier for tracking and analytics (optional) |

**Usage Example**

```typescript
const config: SwapConfig = {
  allowanceStrategy: 'permit',
  slippageBps: 300, // 3% maximum slippage
  stopLimit: '99.00', // Minimum 99 tokens output
  customFee: {
    value: '0.1', // 0.1 token fee
    recipientAddress: '0x1234567890123456789012345678901234567890'
  },
  kitKey: 'KIT_KEY:my-app:secret123'
}
```

---

### BridgeConfig

Configuration options for bridge operations.

```typescript
interface BridgeConfig {
  transferSpeed?: 'FAST' | 'SLOW'
  maxFee?: string
  customFee?: CustomFee
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| transferSpeed | `'FAST'` \| `'SLOW'` | Transfer speed mode (optional, defaults to `'FAST'`) |
| maxFee | string | Maximum fee for FAST transfers in human-readable format. If exceeded, falls back to SLOW. |
| customFee | [CustomFee](#customfee) | Custom developer fee added on top of transfer amount |

**Usage Example**

```typescript
// Fast transfer (default)
const fastConfig: BridgeConfig = {
  transferSpeed: 'FAST'
}

// Fast transfer with max fee limit
const limitedConfig: BridgeConfig = {
  transferSpeed: 'FAST',
  maxFee: '5' // Max 5 USDC fee, falls back to SLOW if exceeded
}

// Slow transfer with custom fee
const customConfig: BridgeConfig = {
  transferSpeed: 'SLOW',
  customFee: {
    value: '1', // 1 USDC developer fee
    recipientAddress: '0x1234567890123456789012345678901234567890'
  }
}
```

---

### CustomFee

Custom fee configuration for developer fee collection.

```typescript
interface CustomFee {
  value?: string
  recipientAddress?: string
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|

| value | string | Absolute fee amount in human-readable format (e.g., `'1'` for 1 USDC) |
| recipientAddress | string | Address to receive 90% of the fee (10% goes to Circle). Must be valid for the source chain. |

---

### AdapterContext

The adapter and its associated chain for operations.

```typescript
type AdapterContext<
  TAdapterCapabilities extends AdapterCapabilities = AdapterCapabilities,
> = {
  adapter: Adapter<TAdapterCapabilities>
  chain: ChainIdentifier
  address?: string // Required for developer-controlled adapters, forbidden for user-controlled
}
```

**Properties**

| Name | Type | Description |
|---|---|---|
| adapter | Adapter | The adapter instance handling blockchain operations |
| chain | ChainIdentifier | Blockchain identifier (string like `'Ethereum'` or ChainDefinition object) |
| address | string | Wallet address (required for developer-controlled adapters, optional for user-controlled) |

**Usage Example**

```typescript
import { createAdapterFromPrivateKey } from '@circle-fin/adapter-viem-v2'

const adapter = createAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as `0x${string}`,
})
```

```
// Use in operations
await kit.bridge({
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.00'
})
```

---

### BridgeDestination

Specifies where tokens should be sent in a bridge operation.

```typescript
type BridgeDestination<TCapabilities> =
  | AdapterContext<TCapabilities>
  | BridgeDestinationWithAddress<TCapabilities>
```

**Types**

| Type | Description |
|------|-------------|
| [AdapterContext](#adaptercontext) | Uses the adapter's default address on the destination chain |
| BridgeDestinationWithAddress | Includes a `recipientAddress` to override the default |

**Usage Example**

```typescript
// Default destination (adapter's address)
await kit.bridge({
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.00'
})

// Custom recipient address
await kit.bridge({
```

```
  from: { adapter, chain: 'Ethereum' },
  to: {
    adapter,
    chain: 'Base',
    recipientAddress: '0x742d35Cc6634C0532925a3b844Bc454e4438f44e'
  },
  amount: '100.00'
})
```

---

## Method Results

### BridgeResult

Result of a completed bridge operation.

```typescript
interface BridgeResult {
  amount: string
  token: 'USDC'
  state: 'pending' | 'success' | 'error'
  config?: BridgeConfig
  provider: string
  source: {
    address: string
    chain: ChainDefinition
  }
  destination: {
    address: string
    chain: ChainDefinition
  }
  steps: BridgeStep[]
}
```

**Properties**

| Name | Type | Description |

|------|------|----------|
| amount | string | Amount transferred |
| token | `'USDC'` | Token transferred |
| state | `'pending'` \| `'success'` \| `'error'` | Current state of the transfer |
| config | BridgeConfig | Configuration used for this transfer |
| provider | string | Bridging provider used (e.g., `'cctpv2'`) |
| source | object | Source chain and address information |
| destination | object | Destination chain and address information |
| steps | BridgeStep[] | Array of executed transaction steps |

**Example**

```json
{
  "state": "success",
  "amount": "100.50",
  "token": "USDC",
  "provider": "cctpv2",
  "source": {
    "address": "0x742d35Cc6634C0532925a3b844Bc454e4438f44e",
    "chain": { "name": "Ethereum", "type": "evm", "chainId": 1 }
  },
  "destination": {
    "address": "0x742d35Cc6634C0532925a3b844Bc454e4438f44e",
    "chain": { "name": "Base", "type": "evm", "chainId": 8453 }
  },
  "steps": [
    {
      "name": "Approve",
      "state": "success",
      "txHash": "0x1234567890abcdef...",
      "explorerUrl": "https://etherscan.io/tx/0x1234567890abcdef..."
    },
    {
      "name": "Burn",
      "state": "success",
      "txHash": "0xabcdef1234567890...",
      "explorerUrl": "https://etherscan.io/tx/0xabcdef1234567890..."
    },
    {
```

```
      "name": "Attestation",
      "state": "success",
      "data": "0x9876543210fedcba..."
    },
    {
      "name": "Mint",
      "state": "success",
      "txHash": "0xfedcba9876543210...",
      "explorerUrl": "https://basescan.org/tx/0xfedcba9876543210..."
    }
  ]
}
```

---

### EstimateResult

Cost estimation result for bridge operations.

```typescript
interface EstimateResult {
  gasFees: {
    name: string
    token: string
    blockchain: Blockchain
    fees: EstimatedGas | null
    error?: unknown
  }[]
  fees: {
    type: 'kit' | 'provider'
    token: 'USDC'
    amount: string | null
    error?: unknown
  }[]
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| gasFees | array | Gas fees for each step on different blockchains |
| fees | array | Protocol and kit fees for the transfer |

**Example**

```json
{
  "gasFees": [
    {
      "name": "Approve",
      "token": "ETH",
      "blockchain": "Ethereum",
      "fees": { "gas": "46000", "gasPrice": "25000000000", "fee": "1150000000000000" }
    },
    {
      "name": "Burn",
      "token": "ETH",
      "blockchain": "Ethereum",
      "fees": { "gas": "120000", "gasPrice": "25000000000", "fee": "3000000000000000" }
    }
  ],
  "fees": [
    { "type": "provider", "token": "USDC", "amount": "0.01" },
    { "type": "kit", "token": "USDC", "amount": "0.50" }
  ]
}
```

---

### BridgeStep

A single step in the bridge process.

```typescript
interface BridgeStep {
  name: string
  state: 'pending' | 'success' | 'error' | 'noop'
```

```
  txHash?: string
  explorerUrl?: string
  data?: unknown
  errorMessage?: string
  error?: unknown
}
```

**Properties**

| Name | Type | Description |
|------|------|-------------|
| name | string | Step name (e.g., `'Approve'`, `'Burn'`, `'Attestation'`, `'Mint'`) |
| state | string | Current state of the step |
| txHash | string | Transaction hash (if applicable) |
| explorerUrl | string | Block explorer URL for the transaction |
| data | unknown | Optional step-specific data (e.g., attestation data) |
| errorMessage | string | Human-readable error message (if failed) |
| error | unknown | Raw error object (if failed) |

---

### SwapResult

Result of a completed swap operation.

```typescript
interface SwapResult {
  tokenIn: SupportedToken
  tokenOut: SupportedToken
  chain: ChainDefinition
  amount: string
  fromAddress: string
  toAddress: string
  config?: SwapConfig
  txHash: string
  explorerUrl?: string
  fees?: readonly ServiceSwapFee[]
}
```

**Properties**

| Name | Type | Description |
|------|------|----------|
| tokenIn | SupportedToken | Input token swapped from |
| tokenOut | SupportedToken | Output token swapped to |
| chain | ChainDefinition | Chain where the swap occurred |
| amount | string | Input amount swapped |
| fromAddress | string | Address that initiated the swap |
| toAddress | string | Address that received the output tokens |
| config | SwapConfig | Configuration used for the swap |
| txHash | string | Transaction hash |
| explorerUrl | string | Block explorer URL |
| fees | array | Fee breakdown for the swap |

**Example**

```json
{
  "tokenIn": "USDC",
  "tokenOut": "USDT",
  "chain": { "name": "Ethereum", "type": "evm", "chainId": 1 },
  "amount": "100.50",
  "fromAddress": "0x742d35Cc6634C0532925a3b844Bc454e4438f44e",
  "toAddress": "0x742d35Cc6634C0532925a3b844Bc454e4438f44e",
  "txHash": "0xabc123...",
  "explorerUrl": "https://etherscan.io/tx/0xabc123...",
  "fees": [
    { "token": "USDC", "amount": "0.05", "type": "provider" }
  ]
}
```

---

### SwapEstimate

Estimation result for a swap operation.

```typescript
interface SwapEstimate {
  readonly transaction: CreateSwapResponse['transaction']
  readonly stopLimit: string
  readonly estimatedOutput?: string
  readonly fees?: readonly ServiceSwapFee[]
}
```

**Properties**

| Name | Type | Description |
|------|------|-----------|
| transaction | object | Raw transaction object to sign and submit |
| stopLimit | string | Minimum output amount in base units (stringified bigint) |
| estimatedOutput | string | Expected output in human-readable format |
| fees | array | Fee breakdown including provider and kit fees |

**Example**

```json
{
  "transaction": { "to": "0xdex...", "data": "0x...", "value": "0x0" },
  "stopLimit": "99450000",
  "estimatedOutput": "99.45",
  "fees": [
    { "token": "USDC", "amount": "0.05", "type": "provider" }
  ]
}
```

---

### EstimatedGas

Gas estimation result for operations.

```typescript
interface EstimatedGas {
  gas: bigint
```

```
  gasPrice: bigint
  fee: string
}
```

**Properties**

| Name | Type | Description |
|---|---|---|
| gas | bigint | Estimated gas units for the transaction |
| gasPrice | bigint | Estimated price per gas unit |
| fee | string | Total estimated fee as a string (gas * gasPrice) |

---

## Supporting Types

### ChainDefinition

Blockchain network configuration.

```typescript
type ChainDefinition = EVMChainDefinition | NonEVMChainDefinition

interface EVMChainDefinition {
  type: 'evm'
  chain: Blockchain
  chainId: number
  name: string
  nativeCurrency: Currency
  isTestnet: boolean
  explorerUrl: string
  rpcEndpoints: readonly string[]
  usdcAddress: string | null
  usdtAddress: string | null
  eurcAddress: string | null
  cctp: CCTPConfig | null
}

interface NonEVMChainDefinition {
```

```
  type: 'solana' | 'algorand' | 'avalanche' | ...
  chain: Blockchain
  name: string
  nativeCurrency: Currency
  isTestnet: boolean
  explorerUrl: string
  rpcEndpoints: readonly string[]
  usdcAddress: string | null
  cctp: CCTPConfig | null
}
```

**Key Properties**

| Name | Type | Description |
|---|---|---|
| type | string | Chain type (`'evm'` or `'solana'`, etc.) |
| chain | Blockchain | Blockchain identifier enum value |
| chainId | number | EVM chain ID (EVM chains only) |
| name | string | Human-readable chain name |
| usdcAddress | string | USDC token contract address |
| usdtAddress | string | USDT token contract address |
| cctp | object | CCTP V2 configuration |

---

### SupportedToken

Tokens supported by StablecoinKit operations.

```typescript
type SupportedToken = 'USDC' | 'USDT' | 'NATIVE'
```

| Value | Description |
|---|---|
| `'USDC'` | USD Coin stablecoin |
| `'USDT'` | Tether stablecoin |
| `'NATIVE'` | Chain's native currency (ETH on Ethereum, SOL on Solana, etc.) |

---

### OperationType

Operation types supported by StablecoinKit.

```typescript
type OperationType = 'bridge' | 'swap'
```

| Value | Description |
|---|---|
| `'bridge'` | Cross-chain USDC transfers via CCTP |
| `'swap'` | Same-chain token swaps |

---

## Event Types

Bridge events are prefixed with `bridge.` to namespace them within StablecoinKit:

| Event | Description |
|---|---|
| `bridge.approve` | Token approval transaction completed |
| `bridge.burn` | Source chain burn transaction completed |
| `bridge.attestation` | CCTP attestation received |
| `bridge.mint` | Destination chain mint transaction completed |
| `*` | Wildcard listener for all events |

**Event Payload**

Each event handler receives a payload with:
- `method` - The action name (e.g., `'approve'`, `'burn'`)
- `values` - Action-specific data including `txHash`, `explorerUrl`, etc.

**Usage Example**

```typescript
const kit = new StablecoinKit()
```

```
// Track progress through bridge steps
kit.on('bridge.approve', (payload) => {
  console.log('Step 1/4: Approval complete')
  console.log('Tx:', payload.values.txHash)
})

kit.on('bridge.burn', (payload) => {
  console.log('Step 2/4: Burn complete')
  console.log('Tx:', payload.values.txHash)
})

kit.on('bridge.attestation', (payload) => {
  console.log('Step 3/4: Attestation received')
})

kit.on('bridge.mint', (payload) => {
  console.log('Step 4/4: Mint complete')
  console.log('Tx:', payload.values.txHash)
})

// Or use wildcard to log all events
kit.on('*', (event) => {
  console.log(`[${event.method}]`, event.values)
})

// Execute bridge
await kit.bridge({
  from: { adapter, chain: 'Ethereum' },
  to: { adapter, chain: 'Base' },
  amount: '100.00'
})
```

# Fees

# App Kit Fees

## Fee Types

There are three general fee types in App Kit:

1. **Protocol Fee**
   - Fee required by the underlying protocol for execution.
   - Example: DEX swap fees, bridging fee
   - Not controlled by App Kit (external cost)
   - Varies by network and action
2. **Developer Fee**
   - Fee added by the integrating developer to monetize the verb.
   - Configurable (fixed, percentage, or both)
   - Optional — only charged if configured by the developer
3. **Circle Fee**
   - Fee charged by Circle for providing infrastructure and execution services.
   - 10% of the configured developer fee, it will be 0 if no developer fee is configured

## Fee Values by Action

Each action may include different fee components:

| Action | Protocol Fee | Developer Fee | Circle Fee |
|---|---|---|---|
| **Send** | Not applicable | Not applicable | |
| **Swap** | Yes<br>● 2 Bips (0.02%) of the swap amount as service provider fee | Optional Developer Fee<br>● Developer fee will be collected in the form of mainstream stablecoins or gas tokens of the blockchain to reduce volatility risk<br>● Circle Fee is 10% of configured developer fee | |
| **Bridge** | Optional<br>● Free for standard bridging | Optional Developer Fee<br>● Circle Fee is 10% of configured developer fee | |

| | | |
|---|---|---|
| | • Typically 0.01 USDC per transfer, refer to this for details | |

## Fee Configuration

Fee configuration and implementation deetails are documented in each respective action page:

- Swap
  - Customize swap developer fee
- Bridge
  - Customize fast bridging and fees
  - Customize bridge developer fee

# [P1] Sample Use Cases

•

# Swap Kit Overview

# Swap with App Kit

Swap is the App Kit action for exchanging supported tokens on the same chain. It reuses your App Kit adapters and lets you control slippage and stop limits to customize the swap experience.

- Walk through the Swap action with the Swap USDT for USDC quickstart.
- Review the supported blockchains and token aliases.
- Estimate the swap rate before performing a swap.

To learn about the other actions available in App Kit, see the App Kit overview.

# Quickstart: Swap USDT to USDC on Ethereum

# Quickstart: Swap USDT to USDC on Ethereum

This quickstart helps you write a script to perform a simple token swap from USDT to USDC on Ethereum.

## Prerequisites

Before you begin, ensure that you've:

- Installed Node.js v22+ and npm.
- Created an Ethereum mainnet wallet and funded it with USDT and ETH tokens.
- Obtained a Kit key from the Circle Developer Console.

## Step 1. Set up the project

This step shows you how to prepare your project and environment.

### 1.1. Set up your development environment

Create a new directory and install Swap Kit and its dependencies:

```shell
# Set up your directory and initialize a Node.js project
mkdir stablecoin-kit-quickstart-USDT-to-USDC
cd stablecoin-kit-quickstart-USDC-to-USDT
npm init -y

# Install Stablecoin Kit and tools
npm install @circle-fin/stablecoin-kit @circle-fin/adapter-viem-v2 viem
typescript tsx dotenv
```

### 1.2. Configure environment variables

Create a .env file in the project directory and add the required keys:

- Replace `{YOUR_PRIVATE_KEY}` with the private key for your Ethereum wallet.
- Replace `{YOUR_KIT_KEY}` with the Kit key that you generated from the Circle Developer Console.

```shell
cat <<'EOF' > .env
PRIVATE_KEY={YOUR_PRIVATE_KEY}
KIT_KEY={YOUR_KIT_KEY}
EOF
```

## Step 2. Perform the swap

This step shows you how to set up your script, execute a swap from USDT to USDC on Ethereum, and check the result.

### 2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script. It transfers 1 USDT to USDC on Ethereum:

```typescript
// Import Stablecoin Kit and its dependencies
import "dotenv/config";
import { StablecoinKit } from "@circle-fin/stablecoin-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { inspect } from "util";

// Initialize the SDK
const kit = new StablecoinKit();

const swapUSDTtoUSDC = async (): Promise<void> => {
  try {
    // Initialize the adapter
    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });

    console.log("---------------Starting Swapping---------------");

    const result = await kit.swap({
      from: { adapter, chain: "Ethereum" },
      tokenIn: "USDT",
      tokenOut: "USDC",
    });
```

```
    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void swapUSDTtoUSDC();
```

## 2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```
TypeScript
npx tsx index.ts
```

## 2.3. Verify the transfer

After the script finishes, find the returned steps array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify the amount matches how much USDC was received.

The following code is an example of how the result of a successful swap might look in the terminal output. The values are used in this example only and are not a real transaction:

```
TypeScript
{
  tokenIn: 'USDT',
  tokenOut: 'USDC',
  chain: {
    type: 'evm',
    chain: 'Ethereum',
    name: 'Ethereum',
    title: 'Ethereum Mainnet',
    nativeCurrency: { name: 'Ether', symbol: 'ETH', decimals: 18 },
    chainId: 1,
```

```
    isTestnet: false,
    explorerUrl: 'https://etherscan.io/tx/{hash}',
    rpcEndpoints: [ 'https://eth.merkle.io', 'https://ethereum.publicnode.com'
],
    eurcAddress: '0x1aBaEA1f7C830bD89Acc67eC4af516284b1bC33c',
    usdcAddress: '0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48',
    usdtAddress: '0xdac17f958d2ee523a2206206994597c13d831ec7',
    cctp: {
      domain: 0,
      contracts: {
        v1: {
          type: 'split',
          tokenMessenger: '0xbd3fa81b58ba92a82136038b25adec7066af3155',
          messageTransmitter: '0x0a992d191deec32afe36203ad87d7d289a738f81',
          confirmations: 65
        },
        v2: {
          type: 'split',
          tokenMessenger: '0x28b5a0e9C621a5BadaA536219b3a228C8168cf5d',
          messageTransmitter: '0x81D40F21F12A8F0E3252Bccb954D722d4c464B64',
          confirmations: 65,
          fastConfirmations: 2
        }
      }
    },
    kitContracts: {
      bridge: '0xB3FA262d0fB521cc93bE83d87b322b8A23DAf3F0',
      adapter: '0x7FB8c7260b63934d8da38aF902f87ae6e284a845'
    }
  },
  amount: '1.00',
  fromAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  toAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  txHash: '0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99d5cccf205897',
  explorerUrl:
'https://etherscan.io/tx/0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99
d5cccf205897',
  fees: [ { token: 'USDT', amount: '0.001', type: 'provider' } ],
  config: {
    kitKey:
'KIT_KEY:fdd99fdbdb3c87b9b3f7b29d6cc17b6e:cc8777b4e419eb6dda9f0de932cf0bb8',
  }
}
```

# Customize Your Swaps

# Estimate Swap Rate

# Estimate Swap Rate

Swap Kit can provide a pre-swap estimate of the amount you'll receive before swapping.

This example estimates how much USDC you'll receive when swapping from 1 USDT:

```typescript
// Set up the swap
const params = {
  from: { adapter, chain: "Ethereum" },
  tokenIn: "USDT",
  amount: "1.00",
  tokenOut: "USDC",
  config: {
     kitKey: process.env.KIT_KEY as string
  }
};

// Estimate swap amount
const estimate = await kit.estimate(params)
console.log(`Estimated output: ${estimate.estimatedOutput}`)

// Proceed to swap
const result = await kit.swap(params);
```

**Note**: An estimate does not guarantee the actual amount you'll receive on a swap, which can vary due to market fluctuations. To avoid receiving less than expected, you can set a slippage tolerance or stop limit.

# Collect a Swap Fee

# Collect a swap fee

Swap Kit lets you collect a swap fee from your end users. You can set this fee globally for the kit or on each swap call. Per-call fees override the global fee. You can set a flat or dynamic fee based on the parameters.

Note: If you use this feature, Circle keeps 10% of the transfer fee you collect from your end users.

## Set fee per swap call

This example configures your call to collect a fee of 0.1 USDT for a specific swap action:

TypeScript

JavaScript

```
const output = await kit.swap({
  from: { adapter, chain: "Ethereum" },
  tokenIn: 'USDT',
  amount: '1.00',
  tokenOut: 'USDC',
  // Collect a fee of 0.1 USDT on this transfer and override the global fee, if
set
  config: {
    kitKey: process.env.KIT_KEY as string,
    customFee: {
      value: '0.1', // 0.1 USDT
      recipientAddress: "YOUR_WALLET_ADDRESS",
    },
  },
})
```

## Set fee globally

This example configures your global setup to collect a fee of 0.1 USDT of the sending token value per swap action:

TypeScript

```typescript
import { StablecoinKit } from '@circle-fin/stablecoin-kit'

// Set fee policy during initialization
const kit = new StablecoinKit({
  customFeePolicy: {
    computeFee: async (params) => {
      // Return fee in human-readable format (e.g., '0.1'
for 0.1 USDT)
      return '0.1'
    },
    resolveFeeRecipientAddress: async (chain, params) => {
      if (chain.type === 'solana') {
        return
'EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v'
      }
      return '0x1234567890123456789012345678901234567890'
    },
  },
})

// Or set/update it later
kit.setCustomFeePolicy({
  computeFee: () => '0.1', // 0.1 unit of the token that
you specified in tokenIn, e.g. 0.1 USDT to be collected as
fee if you are swapping USDT into USDC
  resolveFeeRecipientAddress: (chain) => '0x...',
})

// Remove the fee policy when needed
kit.removeCustomFeePolicy()
```

# Set Slippage Tolerance or Stop Limit

# Set Slippage Tolerance or Stop Limit

Set a slippage tolerance or stop limit to avoid receiving less than expected due to market fluctuations:

- A **slippage tolerance** lets you set the maximum percentage difference you're willing to accept between the estimated and actual swap amount, expressed in bps (for example, 100 bps = 1%).
- A **stop limit** lets you set the exact minimum amount you want to receive.

If both are configured, the stop limit takes precedence.

## Set slippage tolerance

This example sets a slippage tolerance of 100 bps (1%):

JavaScript

```javascript
const output = await kit.swap({
  from: { adapter, chain: "Ethereum" },
  tokenIn: 'USDT',
  amount: '1.00',
  tokenOut: 'USDC',
  config: {
    // Set the splippage tolerance to 100 bps
    slippageBps: 100
  }
})
```

With `slippageBps` as `100`, you receive at least 99% of the estimated amount. For example, if the swap rate is 1 USDT to 1 USDC, you receive at least 0.99 USDC.

**Note**: The `slippageBps` default is 300 bps. `0` represents no tolerance which increase transaction failure rates.

## Set stop limit

`stopLimit` defines the minimum amount of the token type specified in `tokenOut` you receive on a swap.

This example sets a stop limit of 0.95 USDC, meaning you won't receive any less than that on a swap:

```javascript
const output = await kit.swap({
  from: { adapter, chain: "Ethereum" },
  tokenIn: 'USDT',
  amount: '1.00',
  tokenOut: 'USDC',
  config: {
    // Set stop limit to 0.95 USDC
    stopLimit: '0.95',
  }
})
```

# Learn More

# Custom Fees

# Event Handling

# Event Handling

https://developers.circle.com/bridge-kit/references/event-handling

# Error Recovery and Troubleshooting

# Error Recovery and Troubleshooting

https://developers.circle.com/bridge-kit/references/error-recovery-and-troubleshooting

# ❌ Archived docs for reference

# Swap Kit Installation

# Swap Kit Installation

To get started with Swap Kit, install the core package and an adapter that fits your environment.

**Note**: Swap Kit requires a Kit Key. Use the Circle Developer Console to create a developer account and generate a kit key.

## Install Swap Kit

Use your preferred package manager to install the core package:

```shell
# npm
npm install @circle-fin/stablecoin-kit

# npm yarn
yarn add @circle-fin/stablecoin-kit
```

## Install an adapter

Swap Kit supports multiple adapters. You only need to install the one that matches your development environment. Chain client adapters simplify integration with different blockchains. The Circle Wallets adapter uses your Circle Wallets account to connect to developer-controlled wallets and Circle Contracts.

### Install the Viem adapter

Use your preferred package manager to install Circle's Viem adapter and the Viem library for swapping on EVM-compatible blockchains:

```shell
# npm
npm install @circle-fin/adapter-viem-v2 viem

# npm yarn
yarn add @circle-fin/adapter-viem-v2 viem
```

### Install the Ethers adapter

Use your preferred package manager to install Circle's Ethers adapter and the Ethers library for swapping on EVM-compatible blockchains:

```shell
# npm
npm install @circle-fin/adapter-ethers-v6 ethers

# npm yarn
yarn add @circle-fin/adapter-ethers-v6 ethers
```

### Install the Solana adapter

Use your preferred package manager to install Circle's Solana adapter and the Solana library for swapping on Solana:

```shell
# npm
npm install @circle-fin/adapter-solana @solana/web3.js

# npm yarn
yarn add @circle-fin/adapter-solana @solana/web3.js
```

## Install the Circle Wallets adapter

Use your preferred package manager to install the Circle Wallets adapter for swapping on supported EVM chains and Solana.

**Note**: The Circle Wallets adapter requires an API Key and Entity Secret from the Circle Developer Console. These credentials should never be exposed in the browser. For this reason, the adapter is intended for server-side use only.

```shell
# npm
npm install @circle-fin/adapter-circle-wallets

# npm yarn
yarn add @circle-fin/adapter-circle-wallets
```

# Quickstart: Swap USDT to USDC and bridge the token

# Quickstart: Swap USDT to USDC and bridge the token

This quickstart helps you write a script to perform a simple token swap from USDT to USDC on Ethereum and bridge it to Solana.

## Prerequisites

Before you begin, ensure that you've:

- Installed Node.js v22+ and npm.
- Created an Ethereum mainnet wallet and funded it with USDT and ETH tokens.
- Obtained a Kit key from the Circle Developer Console.

## Step 1. Set up the project

This step shows you how to prepare your project and environment.

### 1.1. Set up your development environment

Create a new directory and install Swap Kit and its dependencies:

```shell
# Set up your directory and initialize a Node.js project
mkdir stablecoin-kit-quickstart-USDT-to-USDC
cd stablecoin-kit-quickstart-USDC-to-USDT
npm init -y

# Install Stablecoin Kit and tools
npm install @circle-fin/stablecoin-kit @circle-fin/adapter-viem-v2 viem
typescript tsx dotenv
```

### 1.2. Configure environment variables

Create a .env file in the project directory and add the required keys:

- Replace `{YOUR_PRIVATE_KEY}` with the private key for your Ethereum wallet.
- Replace `{YOUR_KIT_KEY}` with the Kit key that you generated from the Circle Developer Console.

```shell
cat <<'EOF' > .env
PRIVATE_KEY={YOUR_PRIVATE_KEY}
KIT_KEY={YOUR_KIT_KEY}
EOF
```

## Step 2. Perform the swap

This step shows you how to set up your script, execute a swap from USDT to USDC on Ethereum, and check the result.

### 2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script. It transfers 1 USDT to USDC on Ethereum:

```typescript
// Import Stablecoin Kit and its dependencies
import "dotenv/config";
import { StablecoinKit } from "@circle-fin/stablecoin-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { inspect } from "util";

// Initialize the SDK
const kit = new StablecoinKit();

const swapUSDTtoUSDC = async (): Promise<void> => {
  try {
    // Initialize the adapter
    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });

    console.log("---------------Starting Swapping---------------");

    const result = await kit.swap({
      from: { adapter, chain: "Ethereum" },
      tokenIn: "USDT",
      tokenOut: "USDC",
```

```
      amount: "1.00",
      config: {
         kitKey: process.env.KIT_KEY as string
      }
    });

    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void swapUSDTtoUSDC();
```

## 2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```
TypeScript
npx tsx index.ts
```

## 2.3. Verify the transfer

After the script finishes, find the returned steps array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify the amount matches how much USDC was received.

The following code is an example of how the result of a successful swap might look in the terminal output. The values are used in this example only and are not a real transaction:

```
TypeScript
{
  tokenIn: 'USDT',
  tokenOut: 'USDC',
  chain: {
```

```
    type: 'evm',
    chain: 'Ethereum',
    name: 'Ethereum',
    title: 'Ethereum Mainnet',
    nativeCurrency: { name: 'Ether', symbol: 'ETH', decimals: 18 },
    chainId: 1,
    isTestnet: false,
    explorerUrl: 'https://etherscan.io/tx/{hash}',
    rpcEndpoints: [ 'https://eth.merkle.io', 'https://ethereum.publicnode.com'
],
    eurcAddress: '0x1aBaEA1f7C830bD89Acc67eC4af516284b1bC33c',
    usdcAddress: '0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48',
    usdtAddress: '0xdac17f958d2ee523a2206206994597c13d831ec7',
    cctp: {
      domain: 0,
      contracts: {
        v1: {
          type: 'split',
          tokenMessenger: '0xbd3fa81b58ba92a82136038b25adec7066af3155',
          messageTransmitter: '0x0a992d191deec32afe36203ad87d7d289a738f81',
          confirmations: 65
        },
        v2: {
          type: 'split',
          tokenMessenger: '0x28b5a0e9C621a5BadaA536219b3a228C8168cf5d',
          messageTransmitter: '0x81D40F21F12A8F0E3252Bccb954D722d4c464B64',
          confirmations: 65,
          fastConfirmations: 2
        }
      }
    },
    kitContracts: {
      bridge: '0xB3FA262d0fB521cc93bE83d87b322b8A23DAf3F0',
      adapter: '0x7FB8c7260b63934d8da38aF902f87ae6e284a845'
    }
  },
  amount: '1.00',
  fromAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  toAddress: '0x23f9a5BEA7B92a0638520607407BC7f0310aEeD4',
  txHash: '0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99d5cccf205897',
  explorerUrl:
'https://etherscan.io/tx/0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99
d5cccf205897',
  fees: [ { token: 'USDT', amount: '0.001', type: 'provider' } ],
```

```
  config: {
    kitKey:
'KIT_KEY:fdd99fdbdb3c87b9b3f7b29d6cc17b6e:cc8777b4e419eb6dda9f0de932cf0bb8',
  }
}
```

# Swap Draft

DM

***Note:*** *The goal of this document is to share an early draft of the DevDocs for Swap in StablecoinKit, allowing us to review the DevX.*

---

Swap is a simple way for you to trade one token for another. You can use `getSwapQuote()` function to get a quote and make sure you are ok with the exchange rate before calling `swap()`

# Getting Started

### Step 1: Initialization

[Initilialize your StablecoinKit client.](#) You can use a server side wallet like Circle Dev-Controlled Wallets, or client side wallet like Metamask.

### Step 2: Implement your code

To implement the swap function, you need the following code:

```javascript
import { StablecoinKit } from '@circle-fin/stablecoin-kit'
import { createAdapterFromPrivateKey } from
'@circle-fin/adapter-viem-v2'

// Checkout step 1 for other wallet adapters
const chainAdapter = createAdapterFromPrivateKey({ privateKey:
process.env.PRIVATE_KEY, chain: 'Ethereum_Sepolia' })
const kit = new StablecoinKit()


const output = await kit.swap({

  // Supports token aliases (USDC, EURC, USDT) and exact contract address
  tokenIn: 'USDC',
  tokenOut: '0xfff9976782d46cc05630d1f6ebab18b2324d6b14',
```

```
  // Token in amount
  amount: '100',

  // Adapter to sign the transaction
  from: ChainAdapter

})
```

To get a quote without executing a swap, you need the following code:

```javascript
const output = await kit.getSwapQuote({
  tokenIn: 'USDC',
  tokenOut: '0xfff9976782d46cc05630d1f6ebab18b2324d6b14',

  // Token in amount
  amount: '100',

  // The chain to fetch the quote for
  chain: 'Ethereum_Sepolia',
})
```

## Learning More

The following are optional parameters to configure `swap()` function.

### Adding your platform fee

You can optionally specify your platform fees via the following:

```javascript
fees?: { platformBps: number }
```

[TODO: specified fee and where the fee will be collected will be dependent on the DEX aggregator we go with]

## Limiting slippage

Slippage is the difference between the expected price of a trade and the price at which the trade is actually executed. You can optionally limit the slippage in bps via the following:

```javascript
// Default slippage 300 bps
slippageBps?: 10
```

Default slippage is 300 bps. Setting slippageBps to 0 means no tolerance, which can lead to higher failure.

Alternatively you can specify stopLimit as the min amount of tokenOut you would like to receive. If both are provided, the function will respect stopLimit and ignore slippageBps.

```javascript
stopLimit?: '99.5',
```

## Allowance strategy

Allowance strategy applies to ERC20 tokens. It configures the amount of tokenIn allowed to be used in a contract call, in this case swap. When not provided, we will default to a suitable strategy to request permit if it's not already granted. You can optionally provide it for the following setups:

- null: Do not do any allowance actions
- approve-exact: Approves the exact amount needed for the action
- approve-max: Approves the max amount (max uint256)
- permit-exact: Sign a permit for the exact amount needed for the action
- permit-max: Sign a permit for the max amount (max unit256)

```javascript
allowanceStrategy?: undefined | null | 'approve-exact' | 'approve-max' |
'permit-exact' | 'permit-max'
```

Swapping with native token or on Solana does not require allowance strategy configuration.

## Token aliases supported

We support the following token aliases, for all other tokens you can always specify the contract address:

- USDC
- EURC
- USDT
- ETH

# Swap (internal details)

# StablecoinKit T1 verbs design

Common client and account initialization for StablecoinKit

```javascript
const kit = Stablecoinkit()


// Initializing using Circle Wallets
const adapter = CircleWalletsAdapter({
    apiKey, entitySecret, chain
})
const account = await adapter.getAccount({ address: '0x...' })


// Initializing using 3rd party wallet
const adapter = createAdapterFromPrivateKey({
    privateKey: '...private key...',
    chain,
});
// or
const account = privateKeyToAccount('...private key...')
const adapter = ViemChainAdapter({ chain, transport, account })

// Metamask
const adapter = ViemChainAdapter({ chain, transport: window.ethereum })
```

## Swap

### getSwapQuote()

#### Input / Output

```javascript
const output = await kit.getSwapQuote({
  // Token address to swap from
```

```
  // Supports token aliases (USDC, EURC, USDT) and exact contract address
  tokenIn: 'USDC',

  // Token address to swap to
  // Supports token aliases (USDC, EURC, USDT) and exact contract address
  tokenOut: ETH,

  // Token in or out amount depending on `mode`
  amount: '100',

  // Slippage in BPS
  // Difference between the expected price of a trade and the price at which
the trade is actually executed
  // (Optional, default 300). 0 = No tolerance / high failure
  slippageBps?: 300,

  // The chain to fetch the quote for
  chain: 'Ethereum_Sepolia', // ChainDef
})


// Output
// Result is a estimate of the in/out token amounts for the swap
// This object can be directly passed to kit.swap(quote, adapter)
{
  // Token in amount
  amount: '100',

  // Estimated token out amount
  stopLimit?: '99.5',

  // Token address to swap from
  tokenIn: '0xf08a50178dfcde18524640ea6618a1f965821715',

  // Token address to swap to
  tokenOut: '0xfff9976782d46cc05630d1f6ebab18b2324d6b14',

  // The chain the quote is for
  chain: 'Ethereum_Sepolia',
}
```

## Errors

We will return the error from the provider. Common errors include:

- No token pair / No route to token

### MVP consideration

We will only provide a single quote without referencing the provider. We won't have enough providers integrated at MVP to provide a list of quotes, and in the long run we want to validate if there's really a demand for developers to pick from a list of providers, as opposed to them relying on Circle in finding the best quote for them.

## swap()

### Input / Output

```javascript
const output = await kit.swap({
  // Token address to swap from
  // Supports token aliases (USDC, EURC, USDT) and exact contract address
  tokenIn: 'USDC',

  // Token address to swap to
  // Supports token aliases (USDC, EURC, USDT) and exact contract address
  tokenOut: '0xfff9976782d46cc05630d1f6ebab18b2324d6b14',

  // Token in amount
  amount: '100',

  // Slippage in BPS
  // Difference between the expected price of a trade and the price at which
the trade is actually executed
  // (Optional, default 300). 0 = No tolerance / high failure
  slippageBps?: 300,

  // Estimated token out amount
  // (default: Calculate from slippageBps or default to 0)
  stopLimit?: '99.5',

  // Adapter to sign the transaction
  from: ChainAdapter | { adapter: ChainAdapter, chain? ChainDef, account?:
string },



  // StablecoinKit general params
  // Allowance policy (EVM)
```

```
  // Available strategies are dependent on the swap provider
  allowanceStrategy?: undefined | null | 'approve-exact' | 'approve-max |
'permit-exact' | 'permit-max'

  // Platform Fees
  fees?: { platformBps: number },
})


// Outputs
{
  // Transaction hash of the swap transaction
  // May include block confirmed
  txHash: '0xswap-transaction-hash',

  // Array of executed transactions in execution order for the swap
  executedTransactions: [
      {
         txHash: "0xapprove",
         transaction: {
            from: '0xsource-wallet',
             to: '0xtokenIn',
             callData: '...approve...',
             ...
          },
      },
      {
         txHash: "0xswap-transaction-hash",
         transaction: {
            from: '0xsource-wallet',
             to: '0xswap-router',
             callData: '...swap...',
             ...
          },
      },
  ],
}
```

## Examples

JavaScript

```javascript
// Minimal - Default swap mode (EXACT_IN) 100 USDC for USDT with default
slippage (3%)
// I want to swap 100 USDC for at least {CURRENT_PRICE of USDT - 0.03%} USDT
const tx = await kit.swap({
   amount: '100',
   tokenIn: 'USDC',
   tokenOut: 'USDT',
   from,
})


// Quote and Swap - Exact In 100 USDC for USDT with 5% slippage
// I want to swap 100 USDC for at least {CURRENT_PRICE of USDT - 5%} USDT
const quote = await kit.getSwapQuote({
   amount: '100',
   tokenIn: 'USDC',
   tokenOut: 'USDT',
   slippageBps: 500,
})
// Verify if `quote.limit` is acceptable
const tx = await kit.swap({ ...quote, from })


// Direct Exact In swap 100 USDT for MIN 105 USDC
// I want to swap 100 USDT for at least 105 USDC
const tx = await kit.swap({
   amount: '100',
   stopLimit: '105',
   tokenIn: 'USDT',
   tokenOut: 'USDC',
   from,
})


// StablecoinKit general params: Platform fees + allowanceStrategy
const tx = await kit.swap({
   amount: '100',
   tokenIn: 'USDT',
   tokenOut: 'USDC',
   from,
   allowanceStrategy: 'approve-exact',
   fees: { platformBps: 500 },
})
```

### Errors

We will return the error from the provider. Common errors include:

- Source wallet has not approved allowance for the swap
- Exceed slippage threshold
- No token pairs
- Insufficient gas

## Provider decision

This doc captures the detailed decisions we have to make: SWAP option, including the following:

1. In our vision, is it better to integrate with DEX aggregators, or DEXs directly?
    a. Need to review with security on risks of integrating with either, SOC2 compliance implications taking dependencies on 3rd party permissioned services.
    b. Does DEX aggregator indeed provide better liquidity and prices as they claim to? And at what volume, or trading pairs like USDT to USDC?
2. If we choose to go with DEX aggregator, we will need a micro-service that proxies the underlying DEX aggregator(s) with our fee configuration, and open a permissioned API endpoint for StablecoinKit. Permissionless access opens up too many risks.
3. We will need to include fee taker capability. Depends on whether we go with DEXs or DEX aggregators, the integration would be different:
    a. To integrate with DEXs, we need to integrate fee taker with Swap so it's an atomic action (caveats of separate calls captured in the doc above)
    b. To integrate with DEX aggregators, fee can be specified as part of the DEX aggregator API calls.

### Liquidity Comparison (quote)

Permissionless  - api , 200 api call
Permission - 400/1min
QoS for permission and permissionless customer, and also contact with the DEX aggregator once we reach out the rate limit

| | Uniswap (v3 only)<br>EVM | 1inch (classic)<br>EVM | li.fi<br>EVM + Solana | Jupiter<br>Solana | 0x<br>EVM | Raydium<br>Solana | Orca<br>Solana | Uniswap (aggregated) | 1inch (fusion) |
|---|---|---|---|---|---|---|---|---|---|

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1k USDC/USDT | 998 | 998 | 995 | 999 | 988 | 999 | 999 | | |
| 1k USDT/USDC | 995 | 999 | 995 | 1000 | 988 | 1000 | 1000 | | |
| 10k USDC/USDT | 9978 | 9988 | 9948 | 9998 | 9883 | 9997 | 9997 | | |
| 10k USDT/USDC | 9957 | 9991 | 9952 | 10001 | 9886 | 10000 | 10000 | | |
| | | | | | | | | | |
| 5m USDC/USDT | 4999729 | 4995466 | 4974035 | 4996098 | 4941166 | 720066 | Error. subopt. | | |
| 5m USDT/USDC | 4998137 | 4994240 | 4963153 | 4999724 | 4942770 | 4998598 | Error. subopt. | | |
| 15m USDC/USDT | Error. LP | 14985705 | 14921521 | 14959112 | 14821468 | 796986 | Error. subopt. | | |
| 15m USDT/USDC | Error. LP | 14981959 | 14926368 | 14983767 | 14826718 | 796063 | Error. subopt. | | |
| 30m USDC/USDT | Error. LP | 29970337 | 29842830 | 29845305 | 29632694 | 818855 | Error. subopt. | | |
| 30m USDT/USDC | Error. LP | 29961626 | 29852350 | 29877439 | 29650838 | 817869 | Error. subopt. | | |
| 500m USDC/USDT | Error. LP | 310091396 | Error. 404 route | 20641671 | 48003958 | 840535 | Error. subopt. | | |
| 500m USDT/USDC | Error. LP | 232956351 | Error. 404 route | 6136322 | 206168161 | 839485 | Error. subopt. | | |
| 1b USDC/USDT | Error. LP | 286101154 | Error. 404 route | 20645927 | 48004167 | 841250 | Error. subopt. | | |
| 1b USDT/USDC | Error. LP | 230857356 | Error. 404 route | 6136313 | 208733889 | 840194 | Error. subopt. | | |
| | | | | | | | | | |
| 5m USDT/USDC + 0.5% | | | | | | | | 4998840 | 4973392 |
| 5m USDC/USDT + 0.5% | | | | | | | | 4998770 | 4975307 |
| 5m USDT/USDC + 1% | | | | | | | | 4999140 | 4949668 |
| 5m USDC/USDT + 1% | | | | | | | | 4999740 | 4950443 |
| | | | | | | | | | |
| 5m USDC/USDe + 0.5% | | | | | | | | 4995480 | 4970935 |
| 5m USDe/USDC + 0.5% | | | | | | | | 5002410 | 4977770 |
| 5m USDC/USDe + 1% | | | | | | | | 4995360 | 4946069 |
| 5m USDe/USDC + 1% | | | | | | | | 5002480 | 4974999 |
| | | | | | | | | | |
| 5m USDC/DAI + 0.5% | | | | | | | | 4998920 | 4974999 |
| 5m DAI/USDC + 0.5% | | | | | | | | 4999910 | 4974999 |
| 5m USDC/DAI + 1% | | | | | | | | 4998920 | 4949999 |
| 5m DAI/USDC + 1% | | | | | | | | 4999910 | 4974999 |
| | | | | | | | | | |
| 5m USDC/PYUSD + 0.5% | | | | | | | | 224091 | 4973505 |
| 5m PYUSD/USDC + 0.5% | | | | | | | | 521764 | 4973392 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 5m USDC/PYUSD + 1% | | | | | | | | 224091 | 4948512 |
| 5m PYUSD/USDC + 1% | | | | | | | | 521768 | 4974999 |
| | | | | | | | | | |
| 15m USDT/USDC + 0.5% | | | | | | | | 14999000 | 14.920M |
| 15m USDC/USDT + 0.5% | | | | | | | | 14945800 | 14.926M |
| 30m USDT/USDC + 0.5% | | | | | | | | 28566400 | 29.839M |
| 30m USDC/USDT + 0.5% | | | | | | | | 29061700 | 29.851M |

API requests

```shell
// Uniswap v3
// USDCKit swap
const quote = await client.getSwapQuote({
  tokenOut: ETH.contracts.USDC,
  tokenIn: '0xdac17f958d2ee523a2206206994597c13d831ec7',
  tokenInAmount: '1000',
})


// 1inch
curl -v \
    -G \
    -H 'Authorization: Bearer REPLACE_WITH_API_KEY' \
    'https://api.1inch.dev/swap/v6.1/1/swap' \
    -d 'src=0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48' \
    -d 'dst=0xdac17f958d2ee523a2206206994597c13d831ec7' \
    -d 'amount=1000000000' \
    -d 'from=0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045'


// Li.fi
curl -v \
    -G \
    -H 'x-lifi-api-key: REPLACE_WITH_API_KEY' \
    -d 'fromChain=1' \
    -d 'toChain=1' \
    -d 'fromToken=0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48' \
    -d 'toToken=0xdac17f958d2ee523a2206206994597c13d831ec7' \
```

```
    -d 'fromAddress=0xEBe51CC2362023c66d4394fBB25674DD911Daf09' \
    -d 'fromAmount=5000000000000' \
    https://li.quest/v1/quote


// Orca
curl -v \
    -G \
    'https://pools-api.mainnet.orca.so/swap-quote' \
    -d 'to=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v' \
    -d 'from=Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB' \
    -d 'amount=1000000000' \
    -d 'isLegacy=false' \
    -d 'amountIsInput=true' \
    -d 'includeData=true' \
    -d 'includeComputeBudget=false' \
    -d 'maxTxSize=1180'


// 0x
curl -v \
    -G \
    -H '0x-api-key: REPLACE_WITH_API_KEY' \
    -H '0x-version:v2' \
    'https://api.0x.org/swap/permit2/quote' \
    -d 'chainId=1' \
    -d 'sellToken=0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48' \
    -d 'buyToken=0xdac17f958d2ee523a2206206994597c13d831ec7' \
    -d 'sellAmount=1000000000' \
    -d 'taker=0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045'


// Raydium
curl -v \
    -G \
    'https://transaction-v1.raydium.io/compute/swap-base-in' \
    -d 'txVersion=V0' \
    -d 'inputMint=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v' \
    -d 'outputMint=Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB' \
    -d 'amount=10000000000' \
    -d 'slippageBps=50'


// Jupiter
```

```
// https://dev.jup.ag/docs/api/swap-api/quote
curl -v \
    -G \
    'https://lite-api.jup.ag/swap/v1/quote' \
    -d 'inputMint=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v' \
    -d 'outputMint=Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB' \
    -d 'amount=1000000000' \
    -d 'restrictIntermediateTokens=true'
```

**Fee Integration with DEX aggregators**

[Jupiter]

```javascript
const quoteResponse = await (
  await
fetch('https://lite-api.jup.ag/swap/v1/quote?platformFeeBps=20&...')
).json();
```

[0x]

via Affiliate Fees

```javascript
https://api.0x.org/swap/permit2/quote                 // Request a firm quote
  ?chainId=1                                            // Ethereum Mainnet
  &sellToken=0x6B175474E89094C44Da98b954EedeAC495271d0F // Sell DAI
  &sellAmount=4000000000000000000000                    // Sell amount: 4000 (18 decimal)
  &buyToken=0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeEEeE  // Buy ETH
  &taker=$USER_TAKER_ADDRESS                            // Address that will make the trade
  &swapFeeRecipient=$INTEGRATOR_WALLET_ADDRESS       // Wallet address that should receive the affiliate fees
  &swapFeeBps=100                                    // Percentage of buyAmount that should be attributed as affiliate fees
```

```
  &swapFeeToken=0x6B175474E89094C44Da98b954EedeAC495271d0F // Receive trading
fee in sellToken (DAI)
--header '0x-api-key: [API_KEY]' // Replace with your own API key
--header '0x-version: v2' // API version
```

via swap surplus / positive slippage

```
JavaScript
https://api.0x.org/swap/permit2/quote                // Request a firm quote
  ?chainId=1                                           // Ethereum Mainnet
  &sellToken=0x6B175474E89094C44Da98b954EedeAC495271d0F // Sell DAI
  &sellAmount=4000000000000000000000                   // Sell amount: 4000
(18 decimal)
  &buyToken=0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeEEeE  // Buy ETH
  &taker=$USER_TAKER_ADDRESS                           // Address that will
make the trade
  &tradeSurplusRecipient=$INTEGRATOR_WALLET_ADDRESS     // The recipient of any
trade surplus fees
--header '0x-api-key: [API_KEY]'                     // Replace with your own
API key
--header '0x-version: v2'                            // Replace with your own
API key
```

[1inch](#)
Swap is Exact In and fee is taken from INPUT before swap

```
JavaScript
referrer     string       referrer's address

fee          number       this percentage of src token amount will be sent to
referrer,
                          the rest will be used as input for a swap
                          min: 0; max: 3; default: 0
                          example to set a fee to 1.5%: &fee=1.5
```

[li.fi](#)

https://docs.li.fi/sdk/request-routes#route-options

```
JavaScript
integrator    string no     The identifier of the integrator, usually the dApp or
company
                            name. Ideally, this should be specified when
                      configuring the
                            SDK, but it can also be modified during a request

fee           number no     The integrator fee percentage (e.g., 0.03 represents
a 3%
                            fee). This requires the integrator to be verified.
```

CoWSwap

The Partner Fee per order is calculated in the surplus token of the order (i.e. buy token for sell orders, and sell token for buy orders) and is then converted into WETH using an external price provider at the time of order settlement.

https://docs.cow.fi/cow-protocol/reference/core/intents/app-data

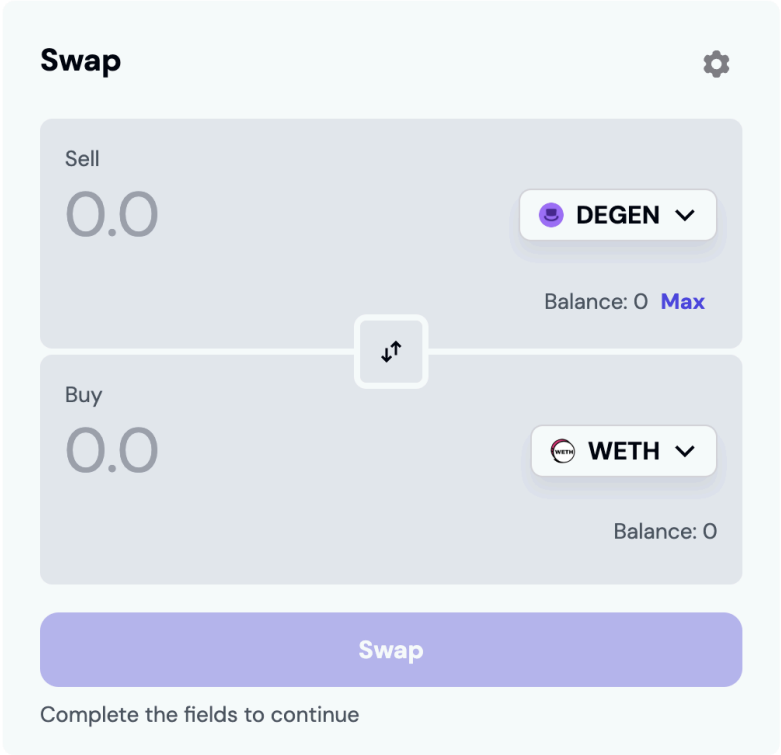
```
JavaScript
// app-data request

partnerFee          object       Partner fee

partnerFee.bps      integer      Basis Point (BPS) required. Possible values :
>= 0 AND
                                 <= 10000.
                                 The fee in basis points (BPS) to be paid to
                            the
                                 partner. One basis point is equivalent to
                            0.01%
                                 (1/100th of a percent)

partnerFee.recipient string      Referrer address required. Pattern:
^0x[a-fA-F0-9]{40}$
```

# Competitor analysis

## Coinbase OnchainKit

Have UI components that developers can specify for swap

```javascript
import { SwapDefault } from '@coinbase/onchainkit/swap';
import type { Token } from '@coinbase/onchainkit/token';

const eth: Token = {
  name: 'ETH',
  address: '',
  symbol: 'ETH',
  decimals: 18,
  image:

'https://wallet-api-production.s3.amazonaws.com/uploads/tokens/eth_288.p
ng',
  chainId: 8453,
};

const usdc: Token = {
  name: 'USDC',
  address: '0x833589fcd6edb6e08f4c7c32d4f71b54bda02913',
  symbol: 'USDC',
  decimals: 6,
  image:

'https://d3r81g40ycuhqg.cloudfront.net/wallet/wais/44/2b/442b80bd16af0c0
d9b22e03a16753823fe826e5bfd457292b55fa0ba8c1ba213-ZWUzYjJmZGUtMDYxNy00ND
cyLTg0NjQtMWI4OGEwYjBiODE2',
  chainId: 8453,
};

<SwapDefault
  from={[eth]}
  to={[usdc]}
/>
```




Also have APIs
[Requires API Key]

`POST /buildSwapTransaction`

JavaScript
```
const response = await buildSwapTransaction({
  fromAddress: '0x...',
  from: fromToken,
  to: toToken,
  amount: '0.1',
  useAggregator: false,
});
```

JSON
```
{
  "approveTransaction": {
    "chainId": 8453,
```

```
    "data": "",
    "gas": 0,
    "to": "",
    "value": 0
  },
  "fee": {
    "baseAsset": {
      "name": "USDC",
      "address": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
      "currencyCode": "USDC",
      "decimals": 6,
      "imageURL":
"https://d3r81g40ycuhqg.cloudfront.net/wallet/wais/44/2b/442b80bd16af0c0
d9b22e03a16753823fe826e5bfd457292b55fa0ba8c1ba213-ZWUzYjJmZGUtMDYxNy00ND
cyLTg0NjQtMWI4OGEwYjBiODE2",
      "blockchain": "eth",
      "aggregators": [],
      "swappable": true,
      "unverified": false,
      "chainId": 8453
    },
    "percentage": "1",
    "amount": "3517825"
  },
  "quote": {
    "from": {
      "address": "",
      "chainId": 8453,
      "decimals": 18,
      "image":
"https://wallet-api-production.s3.amazonaws.com/uploads/tokens/eth_288.p
ng",
      "name": "ETH",
      "symbol": "ETH"
    },
    "to": {
      "address": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
      "chainId": 8453,
      "decimals": 6,
```

```
      "image":
"https://d3r81g40ycuhqg.cloudfront.net/wallet/wais/44/2b/442b80bd16af0c0
d9b22e03a16753823fe826e5bfd457292b55fa0ba8c1ba213-ZWUzYjJmZGUtMDYxNy00ND
cyLTg0NjQtMWI4OGEwYjBiODE2",
      "name": "USDC",
      "symbol": "USDC"
    },
    "fromAmount": "100000000000000000",
    "toAmount": "348264739",
    "amountReference": "from",
    "priceImpact": "",
    "chainId": 8453,
    "highPriceImpact": false,
    "slippage": "3",
    "warning": {
      "type": "warning",
      "message": "This transaction has a very high likelihood of failing
if submitted",
      "description": "failed with 500000000 gas: insufficient funds for
gas * price + value: address 0x4ed4E862860beD51a9570b96d89aF5E1B0Efefed
have 0 want 100000000000000000"
    }
  },
  "transaction": {
    "chainId": 8453,
    "data": "0x...",
    "gas": 419661,
    "to": "0xdef1c0ded9bec7f1a1670819833240f027b25eff",
    "value": 100000000000000000
  },
  "warning": {
    "type": "warning",
    "message": "This transaction has a very high likelihood of failing
if submitted",
    "description": "failed with 500000000 gas: insufficient funds for
gas * price + value: address 0x4ed4E862860beD51a9570b96d89aF5E1B0Efefed
have 0 want 100000000000000000"
  }
}
```

Will separately have to call a sign transaction and broadcast from the response.

`GET /buildSwapTransaction`

JavaScript

```javascript
const quote = await getSwapQuote({
  from: fromToken,
  to: toToken,
  amount: '0.001',
  useAggregator: false,
});
```

JSON


```json
{
  "amountReference": "from",
  "chainId": 8453,
  "from": {
    "address": "",
    "chainId": 8453,
    "decimals": 18,
    "image":
"https://wallet-api-production.s3.amazonaws.com/uploads/tokens/eth_288.p
ng",
    "name": "ETH",
    "symbol": "ETH"
  },
  "to": {
    "address": "0x833589fcd6edb6e08f4c7c32d4f71b54bda02913",
    "chainId": 8453,
    "decimals": 6,
    "image":
"https://d3r81g40ycuhqg.cloudfront.net/wallet/wais/…-ZWUzYjJmZGUtMDYxNy0
0NDcyLTg0NjQtMWI4OGEwYjBiODE2",
    "name": "USDC",
    "symbol": "USDC"
  },
  "fromAmount": "1000000000000000",
  "fromAmountUSD": "2.6519265340000002",
  "toAmount": "2650405",
```

```
  "toAmountUSD": "2.64980125",
  "priceImpact": "0",
  "highPriceImpact": false,
  "slippage": "3"
}
```

## Coinbase CDP

https://docs.cdp.coinbase.com/server-wallets/v1/introduction/quickstart#trading-assets

```javascript
import { Coinbase, Wallet } from "@coinbase/coinbase-sdk";

let coinbase = Coinbase.configureFromJson({ filePath:
'~/Downloads/cdp_api_key.json' });

// Create a Wallet on base-mainnet to trade assets with.
let wallet = await Wallet.create({ networkId: Coinbase.networks.BaseMainnet });

// Trade 0.00001 ETH to USDC.
let trade = await wallet.createTrade({
  amount: 0.00001,
  fromAssetId: Coinbase.assets.Eth,
  toAssetId: Coinbase.assets.Usdc
});
```

https://docs.cdp.coinbase.com/trade-api/quickstart

Coinbase Smart Contract Account

```javascript
https://github.com/coinbase/cdp-sdk/blob/main/examples/typescript/evm/swaps/account.swap.ts

 * Why use account.swap() (all-in-one pattern)?
 * - Simplest developer experience - one function call
 * - Automatically handles quote creation and execution
 * - Manages Permit2 signatures transparently
```

```
 * - Built-in error handling for common issues
 * - Best for 90% of swap use cases
 *
 * This example shows two approaches:
 *
 * Approach 1: All-in-one pattern (RECOMMENDED)
 * - Uses account.swap(...) with inline options
 * - Creates and executes swaps in a single call
 * - Automatically validates liquidity and throws clear errors
 * - Minimal code, maximum convenience
 *
 * Approach 2: Create-then-execute pattern (advanced)
 * - First creates a swap quote using account.quoteSwap()
 * - Allows inspection of swap details before execution
 * - Provides more control for complex scenarios
 * - Use when you need conditional logic based on swap details
```

## Uniswap Swap Widget

https://docs.uniswap.org/sdk/swap-widget/guides/getting-started

```HTML
import { SwapWidget } from '@uniswap/widgets'
import '@uniswap/widgets/fonts.css'

// We recommend you pass a web3 provider.
import { provider } from './your/provider'

// We recommend you pass your own JSON-RPC endpoints.
const jsonRpcUrlMap = {
  1: ['https://mainnet.infura.io/v3/<YOUR_INFURA_PROJECT_ID>'],
  3: ['https://ropsten.infura.io/v3/<YOUR_INFURA_PROJECT_ID>']
}

function App() {
  <div className="Uniswap">
    <SwapWidget provider={provider} jsonRpcUrlMap={jsonRpcUrlMap} />
  </div>
}
```

## SwapKit

https://docs.swapkit.dev/swapkit-api/quote-implementation-flow

```javascript
1. Get Quote
curl -X POST "https://api.swapkit.dev/quote" \
  -H "Content-Type: application/json" \
  -H "x-api-key: YOUR_VARIABLE_HERE" \
  -d '{
    "sellAsset": "BTC.BTC",
    "buyAsset": "ETH.ETH",
    "sellAmount": "0.1",
    "slippage": 3,
    "includeTx": false
  }'

2. Prompt user to review quote

3. Create Swap Transaction (Request is the same as Quote with different
parameters)
curl -X POST "https://api.swapkit.dev/quote" \
  -H "Content-Type: application/json" \
  -H "x-api-key: YOUR_VARIABLE_HERE" \
  -d '{
    "sellAsset": "BTC.BTC",
    "buyAsset": "ETH.ETH",
    "sellAmount": "0.1",
    "sourceAddress": "357a3So9CbsNfBBgFYACGvxxS6tMaDoa1P",
    "destinationAddress": "0x0a4c79cE84202b03e95B7a692E5D728d83C44c76",
    "providers": ["NEAR"], // Filter for the provider
    "slippage": 3,
    "includeTx": true,  // Create tx
  }'


See
https://docs.swapkit.dev/swapkit-api/quote-understanding-the-response#example-r
outes-object
{
    "providers": ["MAYACHAIN"],
    "sellAsset": "ETH.ETH",
    "sellAmount": "0.1",
```

```
    "buyAsset": "BTC.BTC",
    "expectedBuyAmount": "0.00275103",
    "expectedBuyAmountMaxSlippage": "0.00247592",
    "sourceAddress": "0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045",
    "destinationAddress": "357a3So9CbsNfBBgFYACGvxxS6tMaDoa1P",
    "targetAddress": "0xe3985E6b61b814F7Cdb188766562ba71b446B46d",
    "inboundAddress": "0x62b54578b77d0ccfe74b4a009d9ecbc82777c9ff",
    "expiration": "1738931194",
    "memo": "=:b:357a3So9CbsNfBBgFYACGvxxS6tMaDoa1P:247592:_/sk:20/100",
    "fees": [{
        ...
    }],
    "tx": {
        "to": "0xe3985E6b61b814F7Cdb188766562ba71b446B46d",
        "from": "0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045",
        "gas": "0x9714",
        "gasPrice": "0x5be199fd",
        "value": "100000000000000000",
        "data":
"0x44bc937b000000000000000000000000062b54578b77d0ccfe74b4a009d9ecbc82777c9ff0000
000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000016345785d8a000000000000000000000000000000000000
000000000000000000000000000000a000000000000000000000000000000000000000000000000
000000067a5fbfa000000000000000000000000000000000000000000000000000000000000000039
3d3a623a3335376133536f394362734e66424267465941434776787853367 44d61446f6131503a3
234373539323a5f2f736b3a32302f3130300000000000000000"
    },
    "estimatedTime": {
        ...
    },
    "totalSlippageBps": 246,
    "legs": [{
        ...
    }],
    "warnings": [],
    "meta": {
        ...
    }
    "priceImpact": -0.38,
    "approvalAddress": "0xe3985E6b61b814F7Cdb188766562ba71b446B46d",
    }
```

## Trustwallet(use swapkit to provide the SWAP feature)

## 1inch (classic)

1. Uses the 1inch /approve/allowance endpoint to check if your wallet has granted the proper USDC allowance to the 1inch router
    a. If the allowance is insufficient, it uses the /approve/transaction endpoint to construct approval transaction
    b. Send an approval transaction
2. Uses the /swap endpoint to construct the swap transaction
3. Signs and broadcasts the transaction using viem

https://portal.1inch.dev/documentation/apis/swap/classic-swap/quick-start

```javascript
1. HTTP GET /approve/allowance
       { allowance: 123123n }

1a. HTTP GET /approve/transaction
1b. signAndSend(approvalTransaction)

2. HTTP POST /swap

3. signAndSend(swapTransaction)
```

## 1inch (Fusion, intent based)

Submit swap “order” and wait for swap to happen via Dutch auction.

https://portal.1inch.dev/documentation/apis/swap/intent-swap/swagger/relayer?method=post&path=%2Fv2.0%2F1%2Forder%2Fsubmit

```
curl -X POST \
      "https://api.1inch.dev/fusion/relayer/v2.0/1/order/submit" \
      -H "Authorization: Bearer undefined" \
      -H "accept: application/json" \
      -H "content-type: application/json" \
      -d '
```

```
      {
  "order": {
    "salt": "string",
    "makerAsset": "string",
    "takerAsset": "string",
    "maker": "string",
    "receiver": "0x0000000000000000000000000000000000000000",
    "makingAmount": "string",
    "takingAmount": "string",
    "makerTraits": "0"
  },
  "signature": "string",
  "extension": "0x",
  "quoteId": "string"
}'

// 201 Submitted successfully.  Wait for the swap to happen
```

## Orca (Solana DEX)

[https://neonevm.org/docs/composability/using_composability/swap_on_orca#swap-estimation-and-execution](https://neonevm.org/docs/composability/using_composability/swap_on_orca#swap-estimation-and-execution)

- Deprecated SDK
- https://pools-api.mainnet.orca.so/swap-quote?from=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v&to=Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB&amount=5000000000000&isLegacy=false&amountIsInput=true&includeData=true&includeComputeBudget=false&maxTxSize=1180

```javascript
const whirlpool_pubkey = PDAUtil.getWhirlpool(
  ORCA_WHIRLPOOL_PROGRAM_ID,
  DEVNET_WHIRLPOOLS_CONFIG,
  TokenA.mint, // devSAMO
  TokenB.mint, // devUSDC
  64 // tick spacing
).publicKey;

// Token swap pair (devSAMO/devUSDC)
const whirlpool = await client.getPool(whirlpool_pubkey);

const quote = await swapQuoteByInputToken(
    whirlpool,
```

```
    // Token in
    TokenB.mint, // devUSDC
    // Token in amount
    DecimalUtil.toBN(amountIn, TokenB.decimals), // Input Token Mint amount

    // Slippage
    Percentage.fromFraction(10, 1000), // Acceptable slippage (10/1000 = 1%)

    // Extras
    ctx.program.programId,
    ctx.fetcher,
    IGNORE_CACHE
);

console.log("estimatedAmountIn:", DecimalUtil.fromBN(quote.estimatedAmountIn,
TokenB.decimals).toString(), "TokenB");
console.log("estimatedAmountOut:", DecimalUtil.fromBN(quote.estimatedAmountOut,
TokenA.decimals).toString(), "TokenA");
console.log("otherAmountThreshold:",
DecimalUtil.fromBN(quote.otherAmountThreshold, TokenA.decimals).toString(),
"TokenA");

console.log('Executing executeComposabilityMethod with Orca\'s swap instruction
...');
solanaTx = new web3.Transaction();
solanaTx.add(
    WhirlpoolIx.swapIx(
        ctx.program,
        SwapUtils.getSwapParamsFromQuote(
            quote,
            ctx,
            whirlpool,
            ataContractTokenB,
            ataContractTokenA,
            new web3.PublicKey(contractPublicKey)
        )
    )
);

[tx, receipt] = await config.utils.executeComposabilityMethod(
    solanaTx.instructions[0],
    0,
    TestCallSolana,
    undefined,
```

```
    user1
);
console.log(tx, 'tx');
console.log(receipt.logs[0].args, 'receipt args');
```

## Appendix

- Uniswap
  - [Uniswap v3 Slippage formula]

```javascript
const { request } = await writeContract({
  address: router.address,
  abi: router.abi,
  functionName: 'exactInputSingle',
  args: [
    {
      tokenIn: tokenIn.address,
      tokenOut: tokenOut.address,
      fee,
      recipient: account.address,
      amountIn,
      amountOutMinimum,
      sqrtPriceLimitX96,
    },
  ],
  account,
})
```

- Raydium

```javascript
const { data: swapResponse } = await axios({
  method: 'GET',
  url: 'https://transaction-v1.raydium.io/compute/swap-base-in',
  params: {
    inputMint: tokenIn,
```

```
      outputMint: tokenOut,
      amount: tokenInAmount,
      slippageBps: slippage * 100,
      txVersion,
    },
  })

  const { data: responseCreateSwap } = await axios<{
    id: string
    version: string
    success: boolean
    data: { transaction: string }[]
  }>({
    method: 'POST',
    url: 'https://transaction-v1.raydium.io/transaction/swap-base-in',
    data: {
      txVersion,
      computeUnitPriceMicroLamports: computeUnitPriceMicroLamports.toString(),
      wallet: owner.address,
      inputAccount: tokenInATA,
      outputAccount: tokenOutATA,
      swapResponse,
      wrapSol: tokenIn === NATIVE,
      unwrapSol: tokenOut === NATIVE, // true means output mint receive sol,
false means output mint received wsol
    },
  })

account.sign(responseCreateSwap.data.transactions)
sendTransaction()
```

- Jupiter

```javascript
const quoteResponse = await (
  await fetch(

"https://quote-api.jup.ag/v6/quote?inputMint=So11111111111111111111111111111111
111111112&outputMint=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v&amount=100&sl
ippageBps=50",
```

```
  )
).json();

const { swapTransaction: swapTransactionBase64 } = await (
  await fetch("https://quote-api.jup.ag/v6/swap", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      // quoteResponse from /quote API
      quoteResponse,
      // User public key to be used for the swap
      userPublicKey: "FvCt3Jym946xwdk2y24BrUPwDK9VJDRLR4zkKNJmUHpV",
    }),
  })
).json();

account.sign(responseCreateSwap.data.transactions)
sendTransaction()
```

- li.fi

```
JSON
{
  "type": "lifi",
  "id": "1a4e3a47-9428-493b-8d90-d34c1505ac8f:0",
  "tool": "lifidexaggregator",
  "toolDetails": {
    "key": "lifidexaggregator",
    "name": "LI.FI DEX Aggregator",
    "logoURI":
"https://raw.githubusercontent.com/lifinance/types/main/src/assets/icons/exchan
ges/lifidexaggregator.svg"
  },
  "action": {
    "fromToken": {
      "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
      "chainId": 1,
      "symbol": "USDC",
      "decimals": 6,
```

```
      "name": "USD Coin",
      "coinKey": "USDC",
      "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
      "priceUSD": "0.9998123808511938"
    },
    "fromAmount": "1000000",
    "toToken": {
      "address": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
      "chainId": 1,
      "symbol": "USDT",
      "decimals": 6,
      "name": "USDT",
      "coinKey": "USDT",
      "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xdAC17F958D2ee523a2206206994597C13D831ec7/logo.png",
      "priceUSD": "1.0000992691360095"
    },
    "fromChainId": 1,
    "toChainId": 1,
    "slippage": 0.005,
    "fromAddress": "0x552008c0f6870c2f77e5cC1d2eb9bdff03e30Ea0",
    "toAddress": "0x552008c0f6870c2f77e5cC1d2eb9bdff03e30Ea0"
  },
  "estimate": {
    "tool": "lifidexaggregator",
    "approvalAddress": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE",
    "toAmountMin": "990322",
    "toAmount": "995299",
    "fromAmount": "1000000",
    "feeCosts": [
      {
        "name": "LIFI Fixed Fee",
        "description": "Fixed LIFI fee, independent of any other fee",
        "token": {
          "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
          "chainId": 1,
          "symbol": "USDC",
          "decimals": 6,
          "name": "USD Coin",
          "coinKey": "USDC",
```

```
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
          "priceUSD": "0.9998123808511938"
        },
        "amount": "2500",
        "amountUSD": "0.0025",
        "percentage": "0.0025",
        "included": true,
        "feeSplit": {
          "integratorFee": "0",
          "lifiFee": "2500"
        }
      }
    ],
    "gasCosts": [
      {
        "type": "SEND",
        "price": "379855325",
        "estimate": "313633",
        "limit": "407723",
        "amount": "119135165145725",
        "amountUSD": "0.5260",
        "token": {
          "address": "0x0000000000000000000000000000000000000000",
          "chainId": 1,
          "symbol": "ETH",
          "decimals": 18,
          "name": "ETH",
          "coinKey": "ETH",
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2/logo.png",
          "priceUSD": "4415.020336973866"
        }
      }
    ],
    "executionDuration": 30,
    "fromAmountUSD": "0.9998",
    "toAmountUSD": "0.9954"
  },
  "includedSteps": [
    {
      "id": "6db8a9b3-dc97-4de4-8565-ed28682917cb",
```

```
      "type": "protocol",
      "action": {
        "fromChainId": 1,
        "fromAmount": "1000000",
        "fromToken": {
          "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
          "chainId": 1,
          "symbol": "USDC",
          "decimals": 6,
          "name": "USD Coin",
          "coinKey": "USDC",
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
          "priceUSD": "0.9998123808511938"
        },
        "toChainId": 1,
        "toToken": {
          "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
          "chainId": 1,
          "symbol": "USDC",
          "decimals": 6,
          "name": "USD Coin",
          "coinKey": "USDC",
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
          "priceUSD": "0.9998123808511938"
        },
        "fromAddress": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE",
        "toAddress": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE"
      },
      "estimate": {
        "fromAmount": "1000000",
        "toAmount": "997500",
        "toAmountMin": "997500",
        "tool": "feeCollection",
        "approvalAddress": "0xbD6C7B0d2f68c2b7805d88388319cfB6EcB50eA9",
        "gasCosts": [
          {
            "type": "SEND",
            "price": "379855325",
            "estimate": "130000",
            "limit": "169000",
```

```
            "amount": "49381192250000",
            "amountUSD": "0.2180",
            "token": {
              "address": "0x0000000000000000000000000000000000000000",
              "chainId": 1,
              "symbol": "ETH",
              "decimals": 18,
              "name": "ETH",
              "coinKey": "ETH",
              "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2/logo.png",
              "priceUSD": "4415.020336973866"
            }
          }
        ],
        "feeCosts": [
          {
            "name": "LIFI Fixed Fee",
            "description": "Fixed LIFI fee, independent of any other fee",
            "token": {
              "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
              "chainId": 1,
              "symbol": "USDC",
              "decimals": 6,
              "name": "USD Coin",
              "coinKey": "USDC",
              "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
              "priceUSD": "0.9998123808511938"
            },
            "amount": "2500",
            "amountUSD": "0.0025",
            "percentage": "0.0025",
            "included": true,
            "feeSplit": {
              "integratorFee": "0",
              "lifiFee": "2500"
            }
          }
        ],
        "executionDuration": 0
      },
```

```
      "tool": "feeCollection",
      "toolDetails": {
        "key": "feeCollection",
        "name": "Integrator Fee",
        "logoURI":
"https://raw.githubusercontent.com/lifinance/types/main/src/assets/icons/protoc
ols/feeCollection.svg"
      }
    },
    {
      "id": "e20f37b8-451c-4441-93bc-03643881738c",
      "type": "swap",
      "action": {
        "fromChainId": 1,
        "fromAmount": "997500",
        "fromToken": {
          "address": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
          "chainId": 1,
          "symbol": "USDC",
          "decimals": 6,
          "name": "USD Coin",
          "coinKey": "USDC",
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48/logo.png",
          "priceUSD": "0.9998123808511938"
        },
        "toChainId": 1,
        "toToken": {
          "address": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
          "chainId": 1,
          "symbol": "USDT",
          "decimals": 6,
          "name": "USDT",
          "coinKey": "USDT",
          "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xdAC17F958D2ee523a2206206994597C13D831ec7/logo.png",
          "priceUSD": "1.0000992691360095"
        },
        "fromAddress": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE",
        "toAddress": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE"
      },
      "estimate": {
```

```
        "tool": "lifidexaggregator",
        "fromAmount": "997500",
        "toAmount": "995299",
        "toAmountMin": "990322",
        "approvalAddress": "0x8189AFcC5B73Dc90600FeE92e5267Aff1D192884",
        "executionDuration": 30,
        "feeCosts": [],
        "gasCosts": [
          {
            "type": "SEND",
            "price": "379855325",
            "estimate": "60000",
            "limit": "78000",
            "amount": "22791319500000",
            "amountUSD": "0.1006",
            "token": {
              "address": "0x0000000000000000000000000000000000000000",
              "chainId": 1,
              "symbol": "ETH",
              "decimals": 18,
              "name": "ETH",
              "coinKey": "ETH",
              "logoURI":
"https://raw.githubusercontent.com/trustwallet/assets/master/blockchains/ethere
um/assets/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2/logo.png",
              "priceUSD": "4415.020336973866"
            }
          }
        ]
      },
      "tool": "lifidexaggregator",
      "toolDetails": {
        "key": "lifidexaggregator",
        "name": "LI.FI DEX Aggregator",
        "logoURI":
"https://raw.githubusercontent.com/lifinance/types/main/src/assets/icons/exchan
ges/lifidexaggregator.svg"
      }
    }
  ],
  "integrator": "lifi-api",
  "transactionRequest": {
    "value": "0x0",
    "to": "0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE",
```

```
    "data":
"0x5fd9ae2ecaca2db0fe8c2241926055a72d574afe2639eb48d7145b3e81b26180d97496c60000
000000000000000000000000000000000000000000000000000000000000c000000000000000000
000000000000000000000000000000000000000000010000000000000000000000000000552008c0f6
870c2f77e5cc1d2eb9bdff03e30ea000000000000000000000000000000000000000000000000
0000000000f1c720000000000000000000000000000000000000000000000000000000000000160
000000000000000000000000000000000000000000000000000000000000000000086c6966692d61706
900000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000002a3078303030303030303030303030303030303030303
030303030303030303030303030303030303000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000200000000000
000000000000000000000000000000000000000000000000000400000000000000000000000000000
000000000000000000000000000000000000001e000000000000000000000000bd6c7b0d2f68c2b78
05d88388319cfb6ecb50ea900000000000000000000000000bd6c7b0d2f68c2b7805d88388319cfb6
ecb50ea900000000000000000000000000a0b86991c6218b36c1d19d4a2e9eb0ce3606eb480000000
0000000000000000a0b86991c6218b36c1d19d4a2e9eb0ce3606eb480000000000000000000000
000000000000000000000000000000000000000f4240000000000000000000000000000000000000
000000000000000000000000000e000000000000000000000000000000000000000000000000000
000000000001000000000000000000000000000000000000000000000000000000000
```

```
}
```

- Raydium vs Orca Comparison

| | | Raydium | Orca |
|---|---|---|---|
| Trading Volume | 7d | $6.5b (ref) | $4.9b (ref) |
| | 30d | $30.6b (ref) | $19.0b (ref) |
| Top Pair | USDC <-> USDT | 24hr Volume: 2b (ref) | 24hr Volume: 774m (ref) |
| | SOL <-> USDC | 24hr Volume: 103b | 24hr Volume: 211b |
| | SOL <-> USDT | 24hr Volume: 7b | 24hr Volume: 26b |
| Pricing Comparison | 1K USDT -> USDC | 1,000.5 USDC | 1,000.5 USDC |
| | 1M USDT -> USDC | 1,000,302 USDC | 997,848 USDC |
| Technical Details | Testnet Availability | Yes (ref) | Yes (ref) |

## DEX Aggregator Comparisons

| Aggregator | Swap Modes | Dev Fees | Liquidity | Token Pairs | Chains | Volume (24h/30d) | Security | SLA |
|---|---|---|---|---|---|---|---|---|
| **Jupiter** | Exact-In<br>Exact-Out<br>Multi-hop | Yes – integrators can set a platform fee (Jupiter takes a 2.5% cut of that fee) | Aggregates liquidity from 20+ Solana DEXs (e.g. Orca, Raydium) for best prices | Any SPL tokens | Solana only | ≈**$1.0B** / **$30.6B** | Permissionless (non-custodial DeFi protocol)<br><br>Requires API Key for Pro | No formal SLA (best-effort service) |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **0x** | Exact-In<br>Exact-Out<br>Multi-hop | Yes – affiliate fees supported via API params (specify fee % and recipient) | Aggregates 100+ DEX and RFQ sources (Uniswap, Curve, etc.) | Any ERC-20 pairs | 16 EVM chains (Ethereum, BSC, Polygon, Arbitrum, Optimism, Avalanche, Fantom, etc.) | ≈**$0.08B** / **$2.4B** | Permissionless (open protocol)<br><br>Requires API Key | Yes – 99.99% API uptime goal |
| **1inch** | Exact-In<br>Exact-Out<br>Multi-hop | Yes - referrer fee | Aggregates ~522 liquidity sources across 13 chains (Uniswap, Sushi, Curve, etc.) | Any ERC-20 pairs<br><br>(See 1inch intents) | 12 EVM chains<br><br>Arbitrum<br>Avalanche<br>Base<br>Linea<br>BNB Chain<br>Ethereum<br>Gnosis<br>Optimistic Ethereum<br>Polygon<br>zkSync Era<br>Sonic<br><br>(Solana) | ≈**$0.3B** / **$8.8B** | Permissionless (non-custodial)<br><br>Requires API Key | Yes – 99% API uptime (dev portal SLA) |
| **LI.FI** | Exact-In<br>Exact-Out<br>Multi-hop<br>Crosschain | Yes – provides tools for integrators to collect fees (trackable via API) | Meta-aggregator tapping DEX aggregators + bridges (Uniswap, 1inch, Stargate, Across, etc.) | Any asset for swap/bridge | 30+ chains (Ethereum, Solana, Polygon, Arbitrum, Avalanche, Optimism, Base, Bitcoin, Aptos, etc.) | ≈**$0.05B** / **$0.73B** | Permissionless (non-custodial protocol)<br><br>Requires API Key | Yes – enterprise SLAs available for uptime/support |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **CoW Swap** | Limit order swaps | No custom integrator fee (users pay a solver fee via spread; no extra dev fee) | Aggregates on-chain DEX pools + off-chain liquidity via batch auction solvers (can even match trades P2P) | Any ERC-20 pairs | Ethereum Mainnet (+ Gnosis Chain) | ≈**$0.29B** / **$8.6B** | Permissionless (open-source protocol)<br><br>Requires signed appData | No formal SLA (depends on network and solver availability) |

## Solutions

### ➕ SWAP option

#### DEX Aggregator via Circle Proxy

Create a Circle DEX aggregator API that proxies requests to 3rd party DEX aggregators.

- SDK requires a Circle API key

**Flow**: SDK → Circle DEX API → 3rd party DEX Aggregator API

**TODOs**

1. Create Proxy Service to DEX aggregators
    a. HTTP server for handling requests **(3-4 weeks. Dependent on Circle's process to stand up a new service including various reviews)**
        i. Maps incoming requests to DEX aggregator request format
            1. Map SDK method params -> Circle DEX aggregator API params -> 3rd party DEX aggregators
            2. Attach 3rd party DEX aggregator API key
            3. Attach Circle fees
        ii. Map outgoing DEX aggregator responses to Circle DEX format
    b. Add API key support for Circle DEX aggregator **(See estimate below)**
2. Add docs for API key setup **(See estimate below)**
3. Integrate API into SDK **(1 week)**

**Challenges**

- API key will be either confusing if combined with Circle Wallets or yet another Circle API key
- Adding Circle DEX aggregator API
    - Integrate with Circle Wallets API Key
        - Integration 2 weeks, docs should already exist
        - Testnet -> Mainnet upgrade
            - Work to enable console to do an API key without credit card on mainnet
    - ~~Introduce a new Circle API key potentially involving a new Console~~
        - Integration 4 weeks + 2 weeks docs

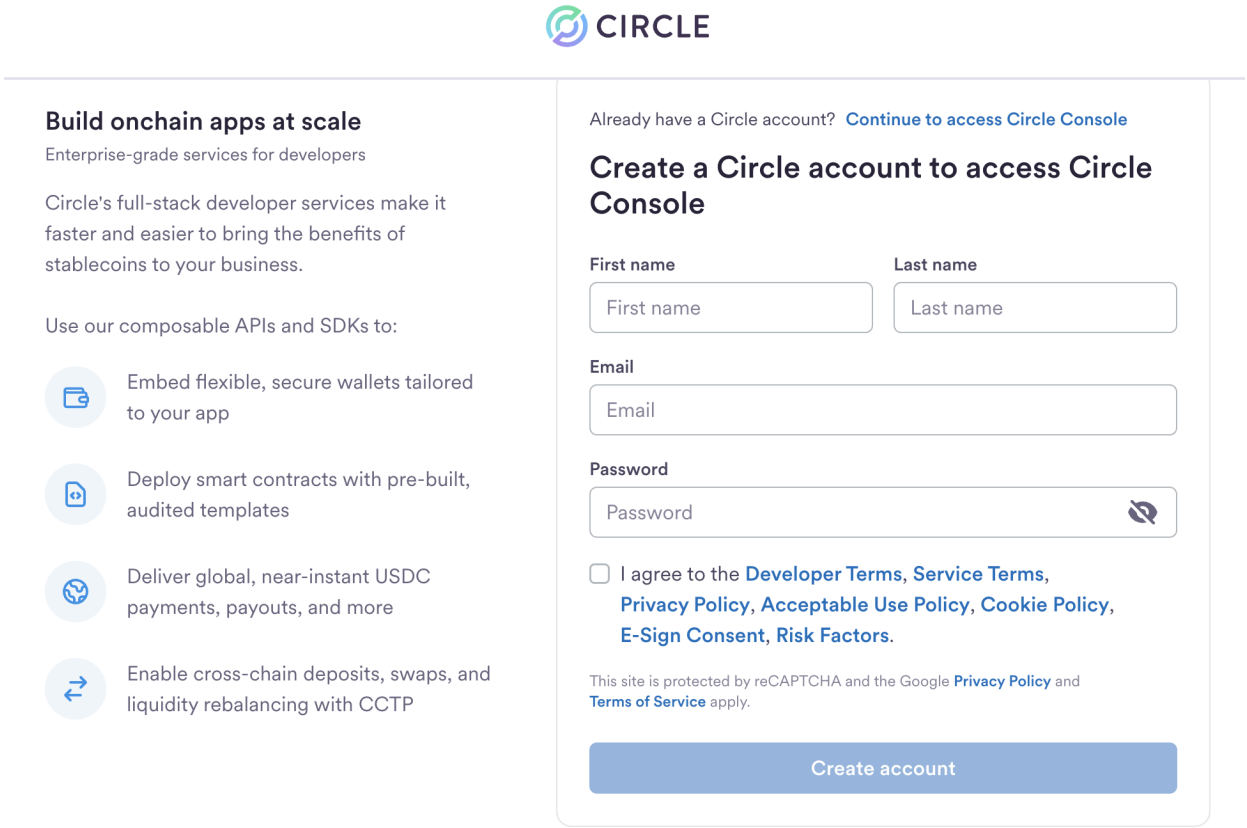
# Kit Key Setup

# Kit Key Setup

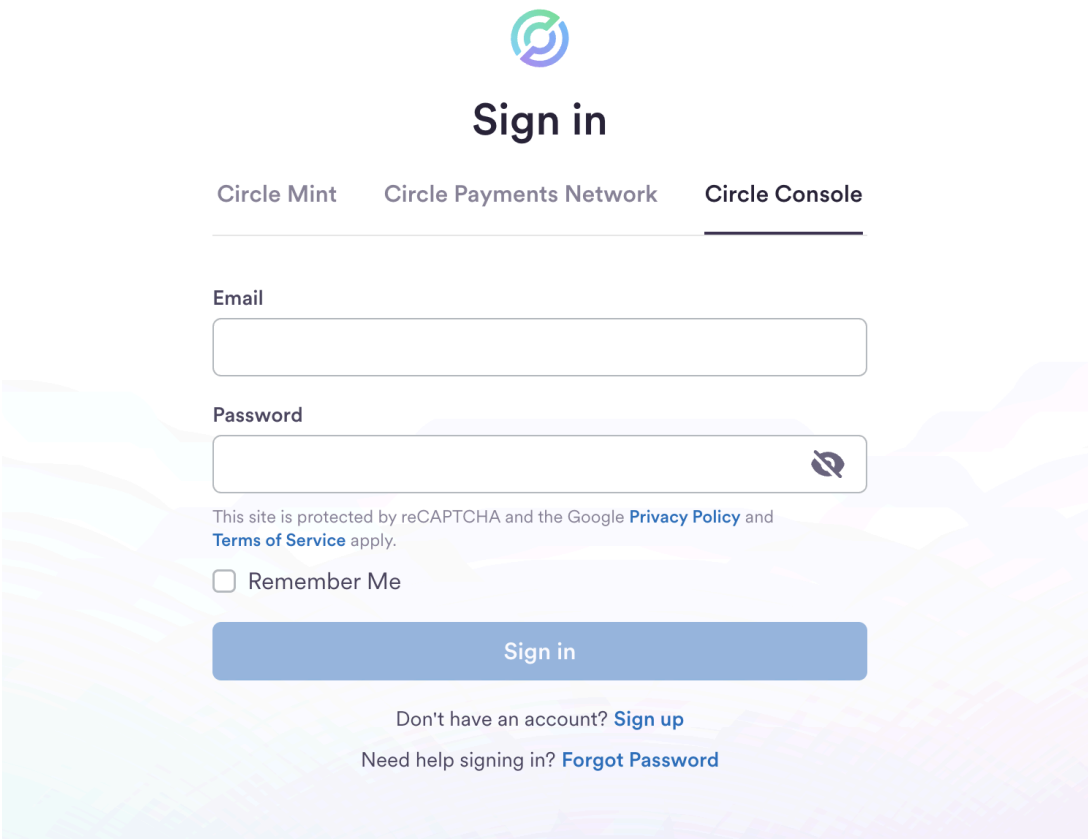
The Stablecoin Kit requires you to create a Kit Key to access the backend swap service provided by Circle. Following the steps below to create a Kit Key:

## 1. Create a Circle Developer Console account

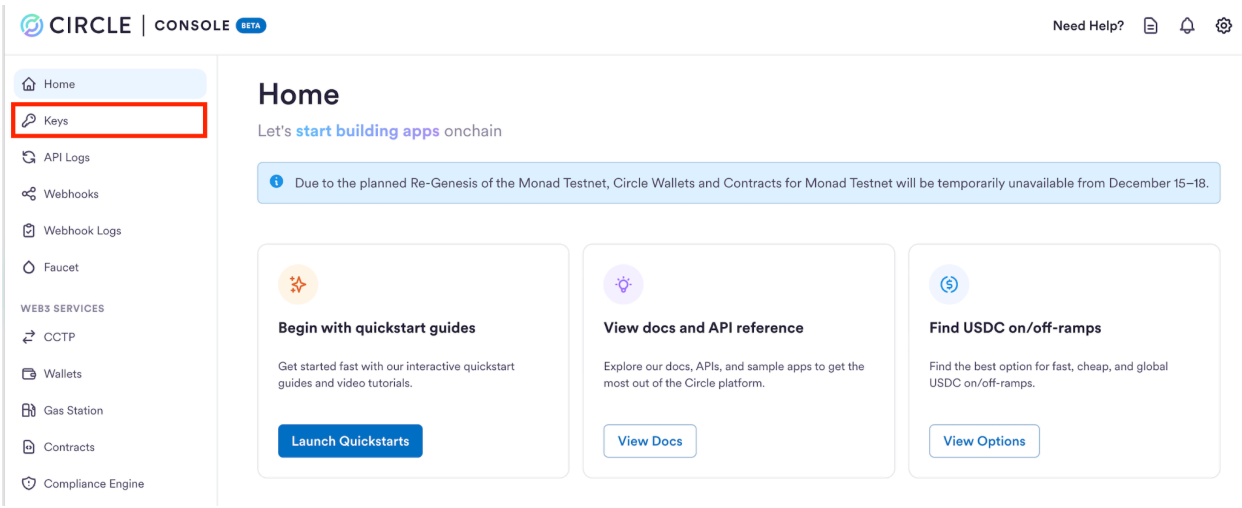
Enter the [Circle Developer Console sign-up page]() to create an account. Skip to the sign-in step if you have an existing account.



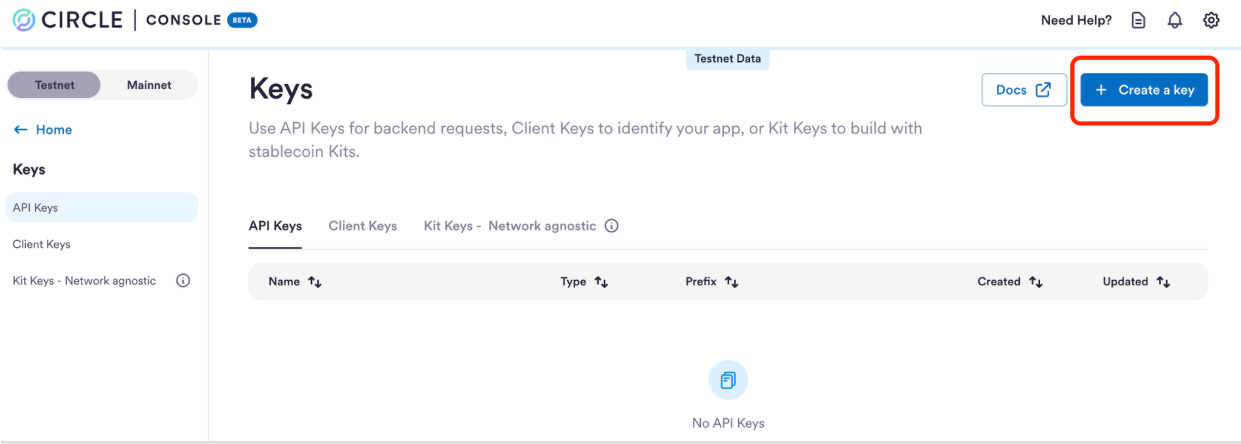
2. Sign in to the Circle Developer Console



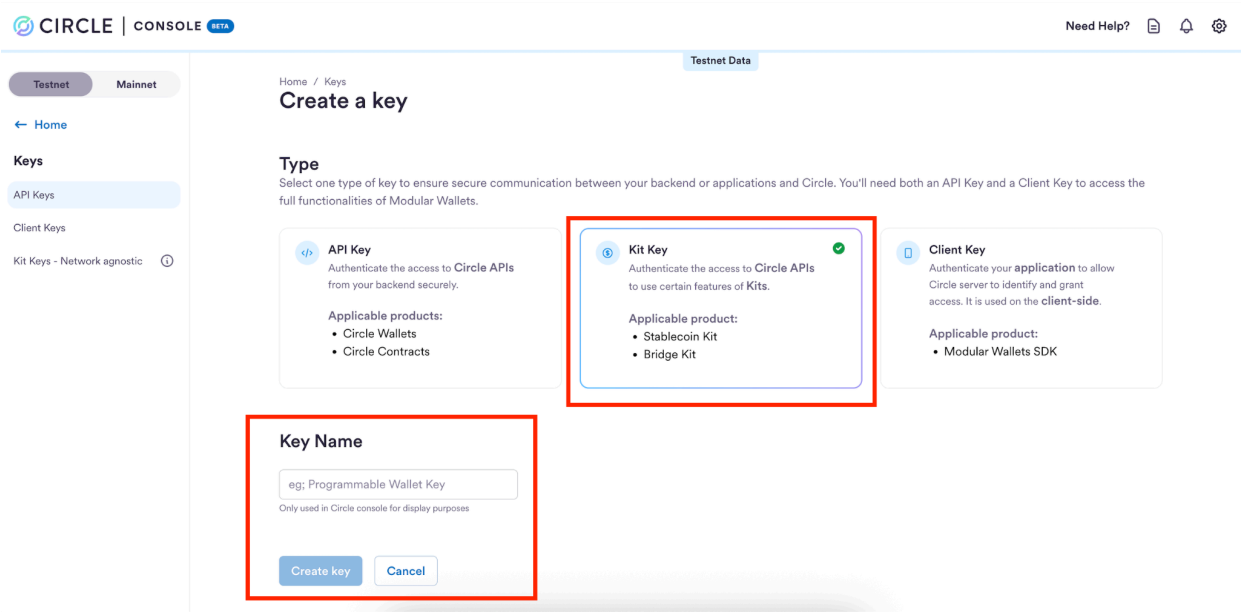

3. Go to *Keys* section



4. Click on *Create a Key*

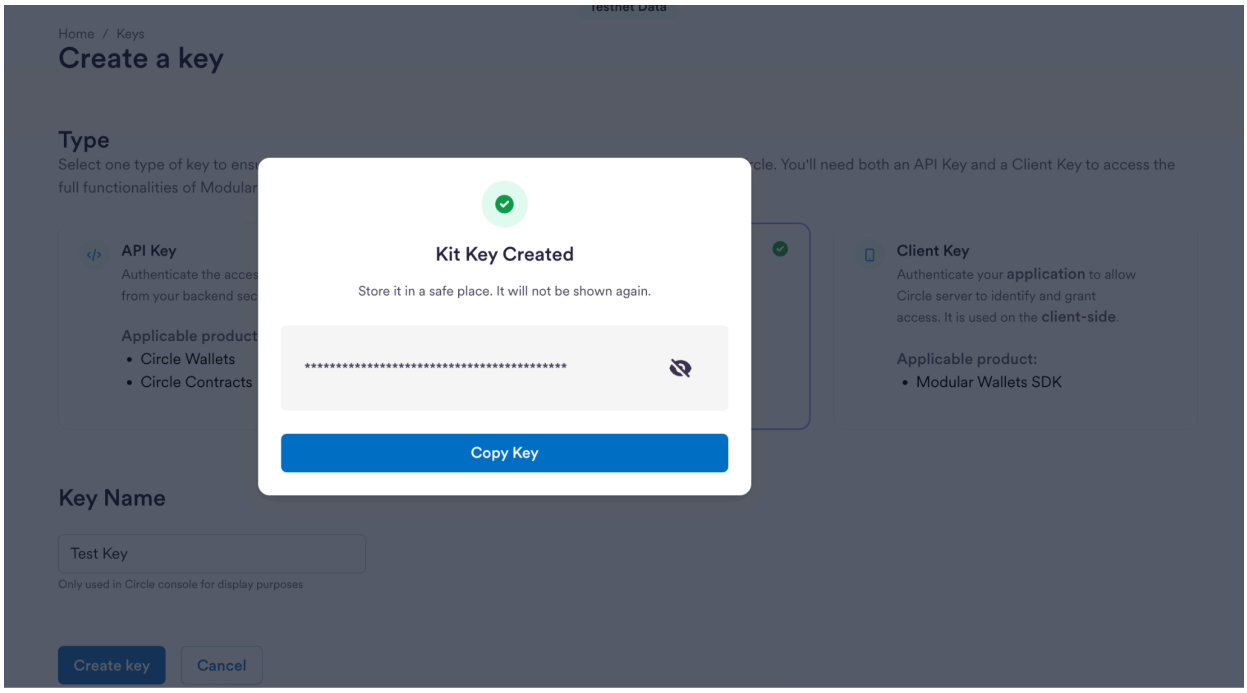



5. Select *Kit Key* and provide *Key Name* to create a key

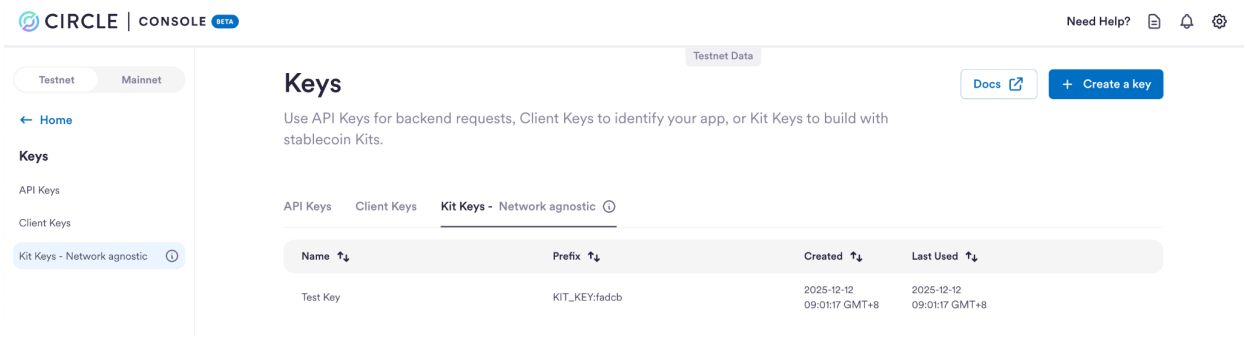



6. Copy and save the Kit Key

Copy and save the Kit Key securely; you will NOT be able to view the key content again.




You will be able to manage the created Kit Key in the *Keys* section once creation is done.




## 7. Configure the Kit Key in your application

You have successfully created and saved the Kit Key! You will need to configure the Kit Key in your Stablecoin Kit to start using it. Refer to the quickstart guide on how to do it:

- [Swap USDT to USDC on Ethereum]